

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. Описание предметной области	5
2. Концептуальное проектирование БД	7
3. Логическое проектирование БД	10
4. Физическая модель БД	12
5. Реализация БД в СУБД PostgreSQL	17
5.1 Создание таблиц	17
5.2 Индексы	19
5.3 Заполнение таблиц данными	19
5.4 Схема базы данных в PostgreSQL	21
6 Разработка запросов к БД	22
6.1. Запрос на выборку данных	22
6.1.1. Однотабличный запрос с простым условием	22
6.1.2. Однотабличные запросы со сложным условием	22
6.1.3. Многотабличный запрос с вычисляемым полем и простым условием	25
6.1.4. Многотабличные запросы со сложными условиями	25
6.2. Запросы с использованием оконных функций	30
6.3. Запросы с вычисляемыми полями	33
6.4. Запросы с агрегатными функциями	35
6.5. Запросы с использованием подзапросов	37
6.6. Запрос на основе материализованного представления	40
7. Разработка объектов БД	42
7.1. Разработка материализованных представлений	42

7.2. Разработка курсоров	45
7.3. Разработка функции.....	47
7.4. Разработка процедуры	49
7.5. Разработка триггеров	51
8. Ограничения целостности БД	55
ЗАКЛЮЧЕНИЕ	57
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	58

ВВЕДЕНИЕ

В современных условиях хозяйственной деятельности предприятий, предоставляющих коммунальные услуги, возрастает необходимость в эффективном управлении процессами учёта, начисления и оплаты услуг. Ручная обработка данных, связанных с потреблением воды, электроэнергии и газа, а также расчётом платежей, является трудоёмкой, подвержена ошибкам и не обеспечивает необходимой оперативности. Внедрение автоматизированных систем на основе баз данных позволяет оптимизировать эти процессы, повысить точность расчётов, обеспечить контроль за платежами и улучшить взаимодействие с потребителями.

Данная работа посвящена проектированию и реализации базы данных для автоматизации учёта оплаты коммунальных услуг, предоставляемых физическим и юридическим лицам. Цель работы — создать структурированное, нормализованное хранилище данных, которое позволит вести учёт клиентов, договоров, приборов учёта, фиксировать показания, рассчитывать начисления на основе актуальных тарифов, регистрировать платежи и автоматически распределять их по неоплаченным счетам.

В процессе работы были решены следующие задачи:

- Проведён анализ предметной области, выделены ключевые сущности и бизнес-правила.
- Разработана концептуальная и логическая модель данных в нотации IDEF1X.
- Реализована физическая модель в СУБД PostgreSQL с созданием таблиц, ограничений целостности и индексов.
- Разработан комплекс запросов различной сложности для анализа данных и формирования отчётности.
- Созданы объекты базы данных: материализованные представления, курсоры, функции, процедуры и триггеры для автоматизации бизнес-процессов.

1. Описание предметной области

Предприятие-поставщик предоставляет физическим и юридическим лицам три вида коммунальных услуг: водоснабжение, электроснабжение и газоснабжение. Для каждого абонента открывается лицевой счёт, к которому привязан один или несколько приборов учёта (счётчиков). Ежемесячно оператор вносит показания приборов учёта в систему; на основании разности текущего и предыдущего показания автоматически рассчитывается объём потребления и сумма к оплате по действующему тарифу. Клиент вносит платежи, которые система распределяет по неоплаченным начислениям.

Физические лица идентифицируются по ФИО и адресу проживания, юридические — по наименованию организации и юридическому адресу. Тарифы могут изменяться со временем; история тарифов сохраняется для корректного перерасчёта за прошлые периоды.

Основные задачи базы данных:

- Учёт клиентов (физических и юридических лиц) и их контактных данных.
- Учёт договоров на оказание коммунальных услуг;
- Регистрация приборов учёта и их привязка к договорам;
- Фиксация ежемесячных показаний приборов учёта;
- Автоматическое начисление сумм к оплате на основе тарифов;
- Регистрация платежей и распределение их по начислениям;
- Формирование отчётов о задолженности и потреблении.

В предметной области действуют следующие бизнес-правила:

- Клиентом может быть физическое лицо (ФИО, паспорт) или юридическое лицо (название, ИНН).
- Каждый клиент может иметь один или несколько договоров на поставку коммунальных услуг.
- Один договор охватывает ровно один вид услуги (водоснабжение, электроснабжение или газоснабжение).

- К каждому договору привязан ровно один прибор учёта; допускается замена прибора с сохранением истории предыдущего.
- Тарифы хранятся с указанием периода действия (дата начала и, опционально, дата окончания). В каждый момент времени для данного вида услуги может быть активен только один тариф.
- Показания вносятся не реже одного раза в месяц. Показание не может быть меньше предыдущего (счётчик не уменьшается, кроме случаев замены).
- Начисление формируется автоматически после внесения показания: $\text{сумма} = (\text{текущее показание} - \text{предыдущее показание}) \times \text{тариф}$, действующий на дату начисления.
- Сумма платежа не может превышать суммарный текущий долг клиента. Платёж распределяется по начислениям в хронологическом порядке.
- Расчётные данные хранятся с точностью: объём потребления — до 0,0001 единицы; тариф и сумма — до 0,01 рубля.

2. Концептуальное проектирование БД

Концептуальная модель отражает ключевые сущности и связи между ними.

Выделенные сущности:

1. **Клиент (Client)** — физические и юридические лица, получающие услуги.
 - Идентификатор клиента (client_id)
 - Тип клиента: «физическое лицо» / «юридическое лицо» (client_type)
 - ФИО / наименование организации (full_name)
 - Контактный телефон (phone)
 - Электронная почта (email)
 - Адрес (address)
2. **Вид услуги (ServiceType)** — справочник предоставляемых ресурсов.
 - Идентификатор вида услуги (service_type_id)
 - Наименование услуги: «Водоснабжение» / «Электроснабжение» / «Газоснабжение» (name)
 - Единица измерения: м³, кВт·ч, м³ (unit)
3. **Тариф (Tariff)** — действующие расценки на единицу потребления с учётом периода действия.
 - Идентификатор тарифа (tariff_id)
 - Ссылка на вид услуги (service_type_id)
 - Цена за единицу, руб. (price_per_unit)
 - Дата начала действия тарифа (valid_from)
 - Дата окончания действия тарифа (valid_to, NULL — тариф действует по сей день)

4. **Договор** (Contract) — соглашение между предприятием и клиентом на поставку определённого вида услуги.

- Идентификатор договора (contract_id)
- Ссылка на клиента (client_id)
- Ссылка на вид услуги (service_type_id)
- Дата заключения договора (start_date)
- Дата расторжения (end_date, NULL — договор действующий)

5. **Прибор учёта** (Meter) — счётчик, установленный в рамках договора.

- Идентификатор прибора (meter_id)
- Ссылка на договор (contract_id)
- Серийный номер прибора (serial_number)
- Дата установки (install_date)
- Начальные показания на момент установки (initial_reading)

6. **Показание** (MeterReading) — ежемесячная запись показания счётчика.

- Идентификатор записи (reading_id)
- Ссылка на прибор учёта (meter_id)
- Значение показания (reading_value)
- Дата снятия показания (reading_date)

7. **Начисление** (Charge) — сформированный счёт к оплате за расчётный период.

- Идентификатор начисления (charge_id)
- Ссылка на договор (contract_id)
- Ссылка на показание, породившее начисление (reading_id)
- Объём потребления (consumption)
- Применённый тариф (tariff_id)
- Сумма к оплате (amount)
- Дата начисления (charge_date)

- Оплаченная сумма (paid_amount)
8. **Платёж** (Payment) — факт поступления денежных средств от клиента.
- Идентификатор платежа (payment_id)
 - Ссылка на клиента (client_id)
 - Сумма платежа (amount)
 - Дата платежа (payment_date)
 - Примечание (note)

3. Логическое проектирование БД

Описание связей между сущностями:

1. Связь «Client → Contract» — неидентифицирующая, тип «один ко многим»: один клиент может иметь несколько договоров на разные виды услуг.
2. Связь «ServiceType → Contract» — неидентифицирующая, тип «один ко многим»: один вид услуги охватывается множеством договоров с разными клиентами.
3. Связь «ServiceType → Tariff» — неидентифицирующая, тип «один ко многим»: для одного вида услуги может существовать несколько тарифов, действовавших в разные периоды.
4. Связь «Contract → Meter» — неидентифицирующая, тип «один ко многим»: в рамках одного договора за всё время могло быть заменено несколько приборов учёта.
5. Связь «Meter → MeterReading» — неидентифицирующая, тип «один ко многим»: с одного прибора снимается множество ежемесячных показаний.
6. Связь «Contract → Charge» — неидентифицирующая, тип «один ко многим»: по каждому договору ежемесячно формируется начисление.
7. Связь «MeterReading → Charge» — неидентифицирующая, тип «один к одному»: каждое начисление порождается конкретным показанием.
8. Связь «Tariff → Charge» — неидентифицирующая, тип «один ко многим»: один тариф может применяться в множестве начислений.
9. Связь «Client → Payment» — неидентифицирующая, тип «один ко многим»: один клиент может совершать множество платежей.

Логическая модель в нотации IDEF1X представлена на рисунке 1.

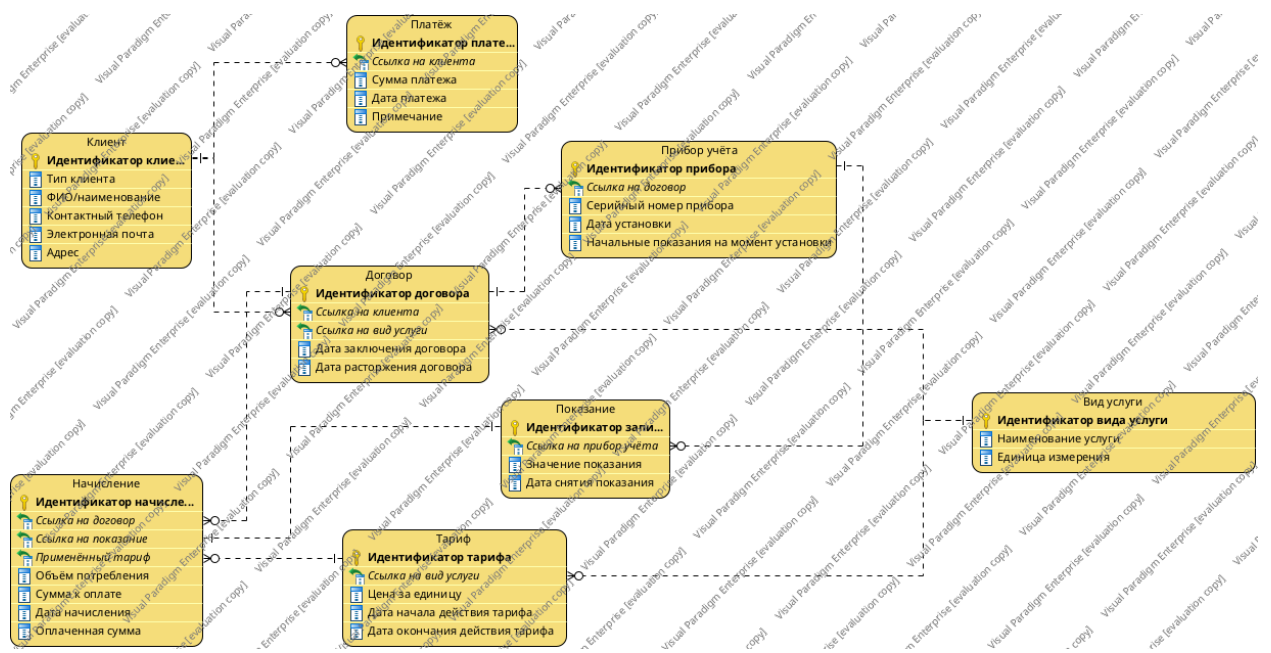


Рисунок 1. – Логическая модель данных

Нормализация:

Все таблицы находятся в третьей нормальной форме (3НФ):

1НФ: Все атрибуты атомарны, нет повторяющихся групп.

2НФ: Все неключевые атрибуты полностью зависят от первичного ключа.

3НФ: Отсутствуют транзитивные зависимости. Все справочные данные (услуги, тарифы) вынесены в отдельные таблицы.

4. Физическая модель БД

Client – Хранит информацию о клиентах предприятия — как физических, так и юридических лицах.

Таблица 1 — Атрибуты Client

Атрибут	Суть	Формат данных	Ограничения
client_id	Уникальный ID клиента	SERIAL	PK, NN
client_type	Тип клиента	VARCHAR(20)	NN, CHECK
full_name	ФИО / Наименование организации	VARCHAR(255)	NN
address	Адрес	TEXT	NN
phone	Контактный телефон	VARCHAR(20)	NN
email	Электронная почта	VARCHAR(100)	

ServiceType – Справочник видов коммунальных услуг, предоставляемых предприятием.

Таблица 2 — Атрибуты ServiceType

Атрибут	Суть	Формат данных	Ограничения
service_type_id	Уникальный ID вида услуги	SERIAL	PK, NN
name	Наименование услуги	VARCHAR(50)	NN, UNIQUE
unit	Единица измерения	VARCHAR(20)	NN

Сущность Tariff – Хранит историю тарифов для каждого вида услуги. Активный тариф — тот, для которого valid_to IS NULL или valid_to >= текущая дата.

Таблица 3 — Атрибуты Tariff

Атрибут	Суть	Формат данных	Ограничения
tariff_id	Уникальный ID тарифа	SERIAL	PK, NN
service_type_id	Ссылка на вид услуги	INT	FK → ServiceType, NN
price_per_unit	Цена за единицу, руб.	DECIMAL(12,4)	NN
valid_from	Дата начала действия тарифа	DATE	NN
valid_to	Дата окончания действия тарифа	DATE	NULL

Сущность Contract – Фиксирует соглашение между предприятием и клиентом на поставку конкретного вида услуги.

Таблица 4 — Атрибуты Contract

Атрибут	Суть	Формат данных	Ограничения
contract_id	Уникальный ID договора	SERIAL	PK, NN
client_id	Ссылка на клиента	INT	FK → Client, NN
service_type_id	Ссылка на вид услуги	INT	FK → ServiceType, NN
start_date	Дата заключения договора	DATE	NN
end_date	Дата расторжения договора	DATE	NULL

Сущность Meter – Прибор учёта (счётчик), установленный по договору. При замене счётчика добавляется новая запись.

Таблица 5 — Атрибуты Meter

Атрибут	Суть	Формат данных	Ограничения
meter_id	Уникальный ID прибора	SERIAL	PK, NN
contract_id	Ссылка на договор	INT	FK → Contract, NN
serial_number	Серийный номер прибора	VARCHAR(50)	NN, UNIQUE
install_date	Дата установки	DATE	NN
initial_reading	Начальные показания	DECIMAL(12,4)	NN

Сущность MeterReading – Ежемесячная запись показания счётчика; является основой для расчёта начисления.

Таблица 6 — Атрибуты MeterReading

Атрибут	Суть	Формат данных	Ограничения
reading_id	Уникальный ID показания	SERIAL	PK, NN
meter_id	Ссылка на прибор учёта	INT	FK → Meter, NN
reading_value	Значение показания	DECIMAL(12,4)	NN
reading_date	Дата снятия показания	DATE	NN

Сущность Charge – Начисленный счёт к оплате. Формируется автоматически триггером при добавлении нового показания.

Поле contract_id хранится явно, несмотря на то что его можно вывести через цепочку reading_id → meter_id → contract_id, — это намеренная денормализация для повышения производительности запросов по задолженности.

Таблица 7 — Атрибуты Charge

Атрибут	Суть	Формат данных	Ограничения
charge_id	Уникальный ID начисления	SERIAL	PK, NN
contract_id	Ссылка на договор	INT	FK → Contract, NN
reading_id	Ссылка на показание	INT	FK → MeterReading, NN, UNIQUE
tariff_id	Ссылка на тариф	INT	FK → Tariff, NN
consumption	Объём потребления	DECIMAL(12,4)	NN
amount	Сумма к оплате, руб.	DECIMAL(12,4)	NN
charge_date	Дата начисления	DATE	NN
paid_amount	Оплаченная сумма, руб.	DECIMAL(12,4)	NN, DEFAULT 0

Сущность Payment – Регистрирует факт поступления денег от клиента. Платёж распределяется по начислениям триггером.

Таблица 8 — Атрибуты Payment

Атрибут	Суть	Формат данных	Ограничения
payment_id	Уникальный ID платежа	SERIAL	PK, NN
client_id	Ссылка на клиента	INT	FK → Client, NN
amount	Сумма платежа, руб.	DECIMAL(12,4)	NN
payment_date	Дата платежа	DATE	NN
note	Примечание	TEXT	NULL

Физическая модель в нотации *IDEF1X* представлена на рисунке 2.

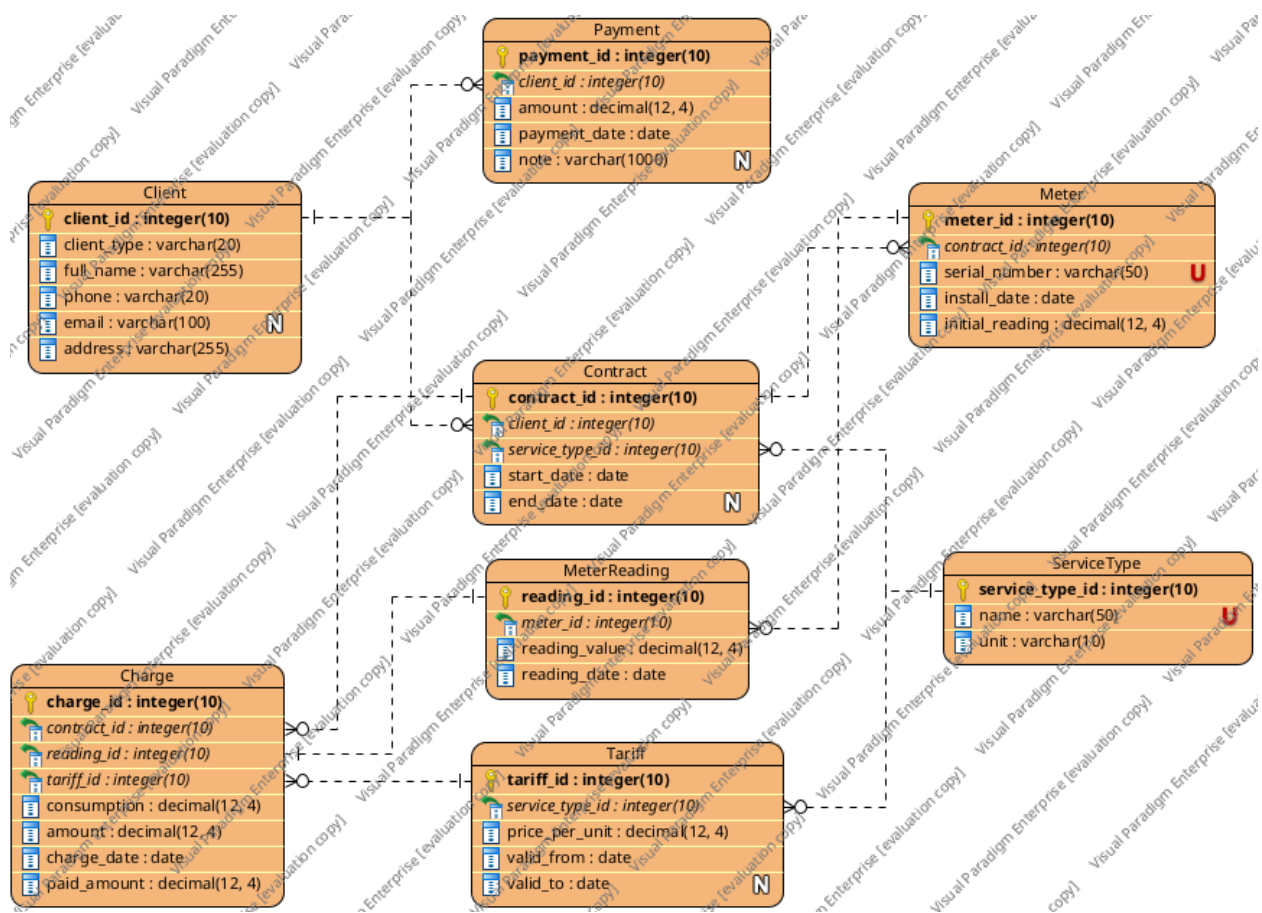


Рисунок 2. – Физическая модель данных

5. Реализация БД в СУБД PostgreSQL

Реализация базы данных выполнена в СУБД PostgreSQL. Для описания структуры базы данных использован язык DDL (Data Definition Language) — подмножество SQL, предназначенное для создания, изменения и удаления объектов базы данных (таблиц, индексов, ограничений). Для заполнения таблиц применён язык DML (Data Manipulation Language) — используется для вставки, обновления и удаления данных.

5.1 Создание таблиц

Для реализации физической модели в СУБД PostgreSQL использован язык определения данных DDL. Ниже представлены операторы CREATE TABLE, создающие 8 таблиц базы данных в соответствии с описанной структурой. Каждая таблица содержит первичный ключ, внешние ключи и необходимые ограничения целостности (NOT NULL, CHECK, UNIQUE).

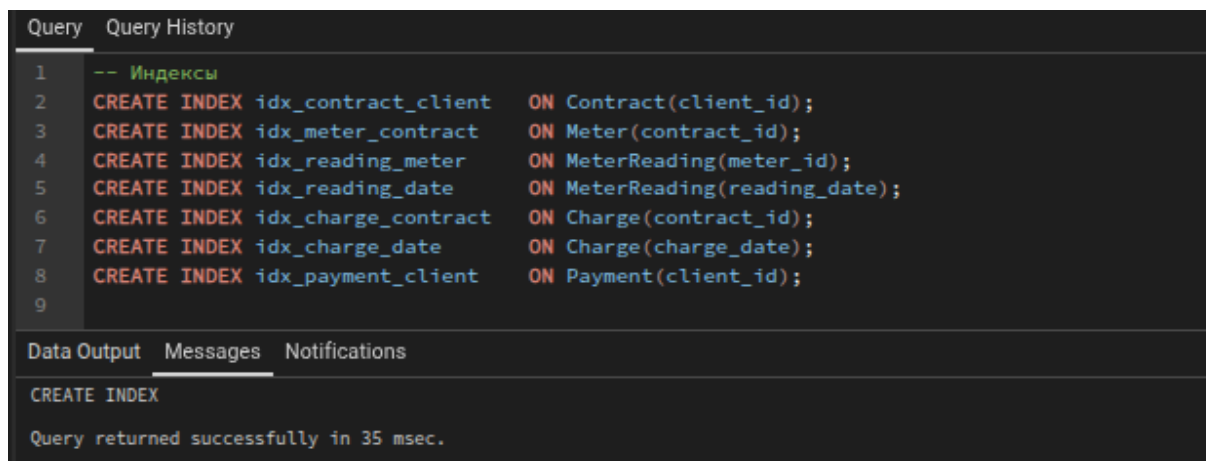
Для всех внешних ключей применяется политика ON DELETE RESTRICT (поведение по умолчанию в PostgreSQL), что запрещает удаление родительской записи при наличии зависимых дочерних записей.


```
Query Query History
1  -- Справочник видов услуг
2  CREATE TABLE ServiceType (
3      service_type_id SERIAL PRIMARY KEY,
4      name            VARCHAR(50) NOT NULL UNIQUE,
5      unit            VARCHAR(20) NOT NULL
6  );
7
8  -- Тарифы
9  CREATE TABLE Tariff (
10     tariff_id        SERIAL PRIMARY KEY,
11     service_type_id INT NOT NULL REFERENCES ServiceType(service_type_id),
12     price_per_unit   DECIMAL(12,4) NOT NULL CHECK (price_per_unit > 0),
13     valid_from       DATE NOT NULL,
14     valid_to         DATE
15 );
16
17 -- Клиенты
18 CREATE TABLE Client (
19     client_id        SERIAL PRIMARY KEY,
20     client_type       VARCHAR(20) NOT NULL CHECK (client_type IN ('физическое', 'юридическое')),
21     full_name         VARCHAR(255) NOT NULL,
22     address           TEXT NOT NULL,
23     phone             VARCHAR(20) NOT NULL,
24     email             VARCHAR(100)
25 );
26
27 -- Договоры
28 CREATE TABLE Contract (
29     contract_id       SERIAL PRIMARY KEY,
30     client_id         INT NOT NULL REFERENCES Client(client_id),
31     service_type_id   INT NOT NULL REFERENCES ServiceType(service_type_id),
32     start_date        DATE NOT NULL,
33     end_date          DATE
34 );
35
36 -- Приборы учёта
37 CREATE TABLE Meter (
38     meter_id          SERIAL PRIMARY KEY,
39     contract_id       INT NOT NULL REFERENCES Contract(contract_id),
40     serial_number     VARCHAR(50) NOT NULL UNIQUE,
41     install_date      DATE NOT NULL,
42     initial_reading   DECIMAL(12,4) NOT NULL
43 );
44
45 -- Показания
46 CREATE TABLE MeterReading (
47     reading_id        SERIAL PRIMARY KEY,
48     meter_id          INT NOT NULL REFERENCES Meter(meter_id),
49     reading_value     DECIMAL(12,4) NOT NULL,
50 );
51
52 Data Output Messages Notifications
53
54 CREATE TABLE
55
56 Query returned successfully in 60 msec.
57
58 Total rows: Query complete 00:00:00.060
```

Рисунок 3. Создание таблиц в PostgreSQL

5.2 Индексы

Для повышения производительности БД и ускорения выполнения наиболее частых запросов было создано 7 индексов с использованием оператора CREATE INDEX. Индексы созданы на внешних ключах и полях дат, которые наиболее часто используются в условиях фильтрации и соединения таблиц.



```
Query    Query History
1      -- Индексы
2      CREATE INDEX idx_contract_client    ON Contract(client_id);
3      CREATE INDEX idx_meter_contract    ON Meter(contract_id);
4      CREATE INDEX idx_reading_meter    ON MeterReading(meter_id);
5      CREATE INDEX idx_reading_date    ON MeterReading(reading_date);
6      CREATE INDEX idx_charge_contract    ON Charge(contract_id);
7      CREATE INDEX idx_charge_date    ON Charge(charge_date);
8      CREATE INDEX idx_payment_client    ON Payment(client_id);
9
Data Output  Messages  Notifications
CREATE INDEX
Query returned successfully in 35 msec.
```

Рисунок 4. Создание индексов в PostgreSQL

5.3 Заполнение таблиц данными

В целях демонстрации работоспособности базы данных выполнено заполнение всех таблиц тестовыми данными. Всего добавлено более 60 записей с помощью оператора INSERT INTO. В таблицу Client внесено 5 клиентов (3 физических и 2 юридических лица), создано 10 договоров, 10 приборов учёта, 18 показаний, 10 начислений и 5 платежей.

Query	Query History												
1	INSERT INTO ServiceType (name, unit) VALUES												
2	('Водоснабжение', 'м³'),												
3	('Электроснабжение', 'кВт·ч'),												
4	('Газоснабжение', 'м³');												
5													
6	INSERT INTO Tariff (service_type_id, price_per_unit, valid_from) VALUES												
7	(1, 45.50, '2024-01-01'),												
8	(2, 5.84, '2024-01-01'),												
9	(3, 8.72, '2024-01-01');												
10													
11	INSERT INTO Client (client_type, full_name, address, phone, email) VALUES												
12	('физическое', 'Иванов Иван Иванович', 'ул. Ленина, 5, кв.10', '79131234567', 'ivanov@')												
13	('физическое', 'Петрова Мария Сергеевна', 'ул. Мира, 12, кв.3', '79135550011', 'petrova@')												
14	('физическое', 'Сидоров Алексей Петрович', 'пр. Победы, 7, кв.21', '79137778899', 'sidorov@')												
15	('юридическое', 'ООО "СтройСервис"', 'ул. Промышленная, 1', '79131112233', 'stroy@biz')												
16	('юридическое', 'АО "ТехноПром"', 'ул. Заводская, 15', '79134445566', 'techno@bt')												
17													
18	INSERT INTO Contract (client_id, service_type_id, start_date) VALUES												
19	(1,1,'2023-01-01'),(1,2,'2023-01-01'),(2,1,'2023-03-01'),												
20	(3,3,'2022-06-01'),(4,1,'2021-09-01'),(4,2,'2021-09-01'),												
21	(5,3,'2022-01-01'),(5,2,'2022-01-01'),(2,3,'2023-05-01'),												
22	(3,2,'2022-08-01');												
23													
24	INSERT INTO Meter (contract_id, serial_number, install_date, initial_reading) VALUES												
25	(1,'WM-0001','2023-01-10',0),(2,'EM-0001','2023-01-10',0),												
26	(3,'WM-0002','2023-03-05',0),(4,'GM-0001','2022-06-15',0),												
27	(5,'WM-0003','2021-09-20',0),(6,'EM-0002','2021-09-20',0),												
28	(7,'GM-0002','2022-01-12',0),(8,'EM-0003','2022-01-12',0),												
29	(9,'GM-0003','2023-05-05',0),(10,'EM-0004','2022-08-01',0);												
30													
31	INSERT INTO MeterReading (meter_id, reading_value, reading_date) VALUES												
32	(1,12.5000,'2024-01-31'),(1,26.1000,'2024-02-29'),(1,41.3000,'2024-03-31'),												
33	(2,310.0000,'2024-01-31'),(2,618.0000,'2024-02-29'),(2,945.0000,'2024-03-31'),												
34	(3,8.0000,'2024-01-31'),(4,55.2000,'2024-01-31'),(5,30.0000,'2024-01-31'),												
35	(6,420.0000,'2024-01-31'),(7,72.4000,'2024-01-31'),(8,380.0000,'2024-01-31'),												
36	(9,18.6000,'2024-01-31'),(10,500.0000,'2024-01-31'),(3,17.5000,'2024-02-29'),												
37	(4,67.0000,'2024-02-29'),(5,58.0000,'2024-02-29'),(6,830.0000,'2024-02-29');												
38													
39	INSERT INTO Charge (contract_id, reading_id, tariff_id, consumption, amount, charge_date)												
40	(1,1,1,12.5000,568.75,'2024-01-31'),												
41	(1,2,1,13.6000,618.80,'2024-02-29'),												
42	(1,3,1,15.2000,691.60,'2024-03-31'),												
43	(2,4,2,310.0000,1810.40,'2024-01-31'),												
44	(2,5,2,308.0000,1798.72,'2024-02-29'),												
45	(2,6,2,327.0000,1909.68,'2024-03-31'),												
46	(3,7,1,8.0000,364.00,'2024-01-31'),												
47	(4,8,3,55.2000,481.34,'2024-01-31'),												
48	(5,9,1,30.0000,1365.00,'2024-01-31'),												
49	(6,10,2,420.0000,2452.80,'2024-01-31');												
50													
51	INSERT INTO Payment (client_id, amount, payment_date, note) VALUES												
52	(1,1200.00,'2024-02-10','Оплата за январь');												
<table border="1"> <thead> <tr> <th>Data Output</th> <th>Messages</th> <th>Notifications</th> </tr> </thead> <tbody> <tr> <td colspan="3">INSERT 0 5</td> </tr> <tr> <td colspan="3">Query returned successfully in 43 msec.</td> </tr> <tr> <td>Total rows:</td> <td colspan="2">Query complete 00:00:00.043</td> </tr> </tbody> </table>		Data Output	Messages	Notifications	INSERT 0 5			Query returned successfully in 43 msec.			Total rows:	Query complete 00:00:00.043	
Data Output	Messages	Notifications											
INSERT 0 5													
Query returned successfully in 43 msec.													
Total rows:	Query complete 00:00:00.043												

Рисунок 5. Заполнение таблиц в PostgreSQL

5.4 Схема базы данных в PostgreSQL

Для наглядного представления структуры разработанной базы данных была построена физическая схема в pgAdmin, отражающая таблицы, их атрибуты, первичные и внешние ключи, а также связи между сущностями. Схема соответствует физической модели из раздела 4.

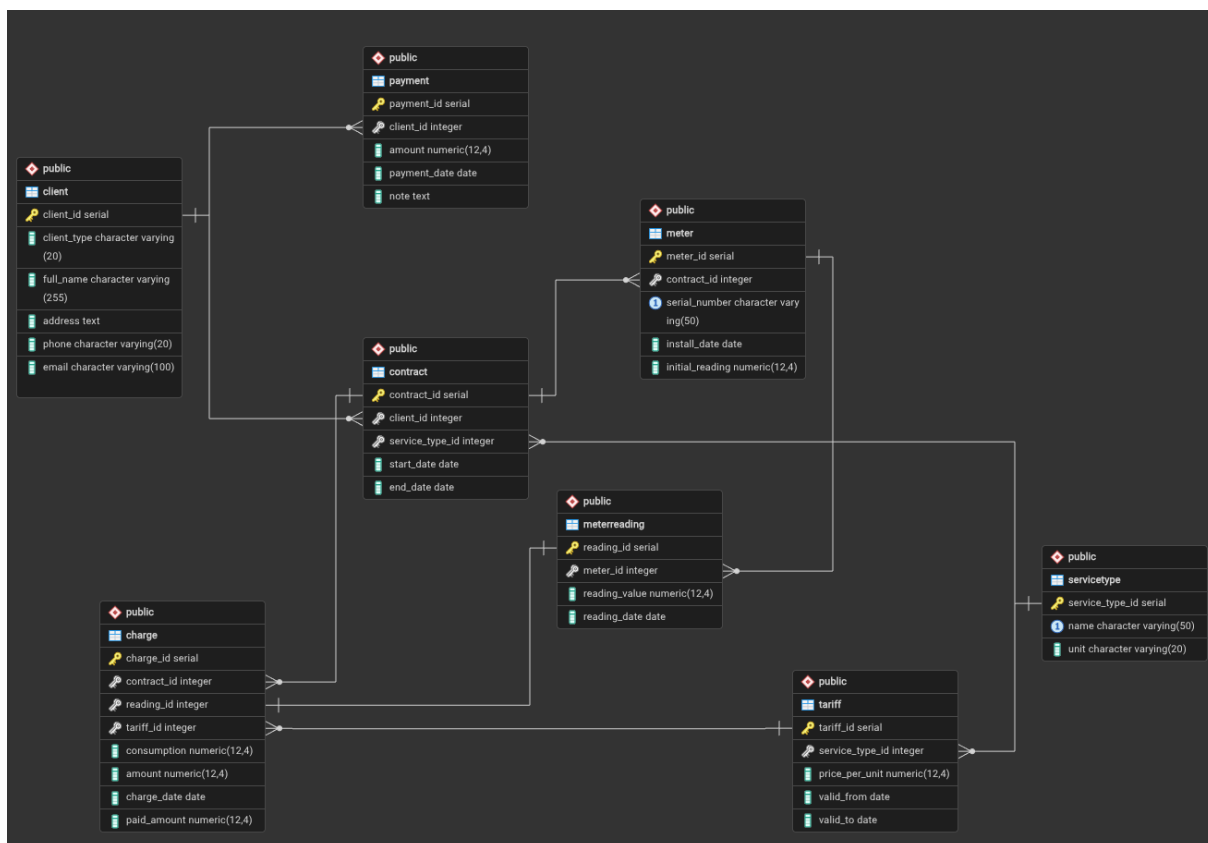


Рисунок 6. Физическая схема базы данных в PostgreSQL

6 Разработка запросов к БД

Для проверки корректности структуры базы данных и демонстрации её аналитических возможностей были разработаны SQL-запросы различных типов в соответствии с заданием. Все запросы выполнены в СУБД PostgreSQL.

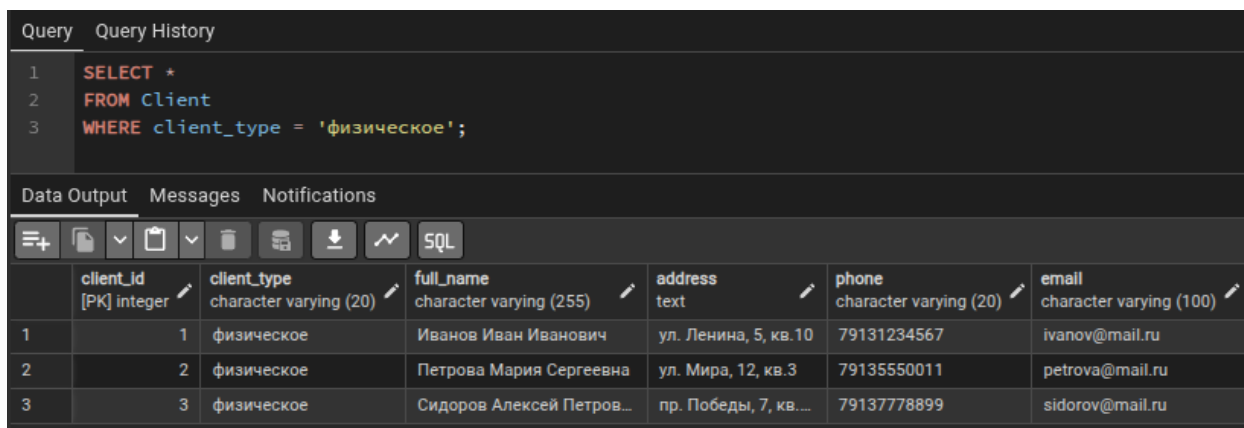
6.1. Запрос на выборку данных

6.1.1. Однотабличный запрос с простым условием

а. Однотабличный запрос с простым условием

Задача: вывести всех клиентов — физических лиц.

Результат выполнения запроса представлен на рисунке 7.



The screenshot shows a PostgreSQL query editor interface. The top panel displays a SQL query: `SELECT * FROM Client WHERE client_type = 'физическое';`. The bottom panel shows the results of the query in a table format. The table has 7 columns: `client_id` (integer), `client_type` (text), `full_name` (text), `address` (text), `phone` (text), and `email` (text). There are 3 rows of data.

	client_id [PK] integer	client_type character varying (20)	full_name character varying (255)	address text	phone character varying (20)	email character varying (100)
1	1	физическое	Иванов Иван Иванович	ул. Ленина, 5, кв.10	79131234567	ivanov@mail.ru
2	2	физическое	Петрова Мария Сергеевна	ул. Мира, 12, кв.3	79135550011	petrova@mail.ru
3	3	физическое	Сидоров Алексей Петров...	пр. Победы, 7, кв....	79137778899	sidorov@mail.ru

Рисунок 7. – Результат выполнения однотабличного запроса с простым условием

6.1.2. Однотабличные запросы со сложным условием

б1. Запрос с условиями AND, AND, NOT:

Задача: вывести начисления за 2024 год с суммой более 500 руб., у которых потребление не равно нулю.

Результат выполнения запроса представлен на рисунке 8.

Query		Query History							
1	SELECT *								
2	FROM Charge								
3	WHERE charge_date >= '2024-01-01'								
4	AND amount > 500								
5	AND NOT consumption = 0;								
6									
Data Output		Messages							
		Notifications							
		SQL							
	charge_id [PK] integer	contract_id integer	reading_id integer	tariff_id integer	consumption numeric (12,4)	amount numeric (12,4)	charge_date date	paid_amount numeric (12,4)	
1	1	1	1	1	12.5000	568.7500	2024-01-31	0.0000	
2	2	1	2	1	13.6000	618.8000	2024-02-29	0.0000	
3	3	1	3	1	15.2000	691.6000	2024-03-31	0.0000	
4	4	2	4	2	310.0000	1810.4000	2024-01-31	0.0000	
5	5	2	5	2	308.0000	1798.7200	2024-02-29	0.0000	
6	6	2	6	2	327.0000	1909.6800	2024-03-31	0.0000	
7	9	5	9	1	30.0000	1365.0000	2024-01-31	0.0000	
8	10	6	10	2	420.0000	2452.8000	2024-01-31	0.0000	

Рисунок 8. – Результат выполнения однотобличного запроса с условием AND, AND, NOT

b2. Запрос с условиями AND, OR:

Задача: вывести начисления за январь 2024 года с суммой более 1000 руб. или потреблением более 50 единиц.

Результат выполнения запроса представлен на рисунке 9.

Query

Query History

1

SELECT *

2

FROM Charge

3

WHERE charge_date >= '2024-01-01'

4

AND (amount > 1000 OR consumption > 50);

5

Data Output

Messages

Notifications

≡+

SQL

	charge_id [PK] integer	contract_id integer	reading_id integer	tariff_id integer	consumption numeric (12,4)	amount numeric (12,4)	charge_date date	paid_amount numeric (12,4)
1	4	2	4	2	310.0000	1810.4000	2024-01-31	0.0000
2	5	2	5	2	308.0000	1798.7200	2024-02-29	0.0000
3	6	2	6	2	327.0000	1909.6800	2024-03-31	0.0000
4	8	4	8	3	55.2000	481.3400	2024-01-31	0.0000
5	9	5	9	1	30.0000	1365.0000	2024-01-31	0.0000
6	10	6	10	2	420.0000	2452.8000	2024-01-31	0.0000

Рисунок 9. – Результат выполнения однотабличного запроса с условием
AND, OR

б3. Условия AND, AND:

Задача: вывести начисления за январь 2024 года, которые ещё не оплачены полностью.

Результат выполнения запроса представлен на рисунке 10.

Query

Query History

1

SELECT *

2

FROM Charge

3

WHERE paid_amount < amount

4

AND charge_date >= '2024-01-01'

5

AND charge_date <= '2024-01-31';

6

Data Output

Messages

Notifications

≡+

SQL

	charge_id [PK] integer	contract_id integer	reading_id integer	tariff_id integer	consumption numeric (12,4)	amount numeric (12,4)	charge_date date	paid_amount numeric (12,4)
1	1	1	1	1	12.5000	568.7500	2024-01-31	0.0000
2	4	2	4	2	310.0000	1810.4000	2024-01-31	0.0000
3	7	3	7	1	8.0000	364.0000	2024-01-31	0.0000
4	8	4	8	3	55.2000	481.3400	2024-01-31	0.0000
5	9	5	9	1	30.0000	1365.0000	2024-01-31	0.0000
6	10	6	10	2	420.0000	2452.8000	2024-01-31	0.0000

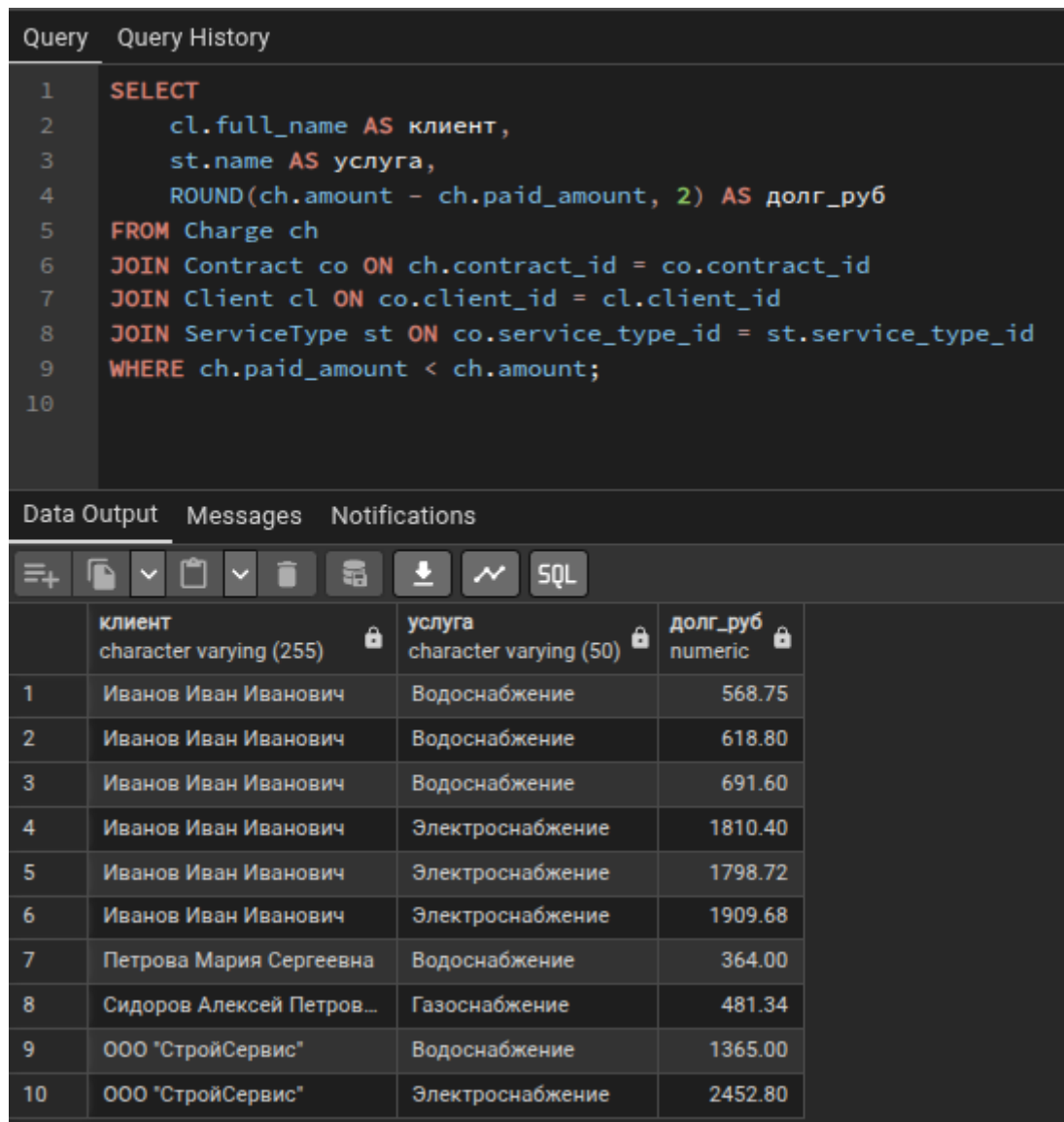
Рисунок 10. – Результат выполнения однотабличного запроса с условием
AND, AND

6.1.3. Многотабличный запрос с вычисляемым полем и простым условием

с. Многотабличный запрос с вычисляемым полем и простым условием

Задача: вывести ФИО клиента, вид услуги и сумму долга по каждому неоплаченному начислению.

Результат выполнения запроса представлен на рисунке 11.



```
1 SELECT
2     cl.full_name AS клиент,
3     st.name AS услуга,
4     ROUND(ch.amount - ch.paid_amount, 2) AS долг_руб
5 FROM Charge ch
6 JOIN Contract co ON ch.contract_id = co.contract_id
7 JOIN Client cl ON co.client_id = cl.client_id
8 JOIN ServiceType st ON co.service_type_id = st.service_type_id
9 WHERE ch.paid_amount < ch.amount;
```

	клиент character varying (255)	услуга character varying (50)	долг_руб numeric
1	Иванов Иван Иванович	Водоснабжение	568.75
2	Иванов Иван Иванович	Водоснабжение	618.80
3	Иванов Иван Иванович	Водоснабжение	691.60
4	Иванов Иван Иванович	Электроснабжение	1810.40
5	Иванов Иван Иванович	Электроснабжение	1798.72
6	Иванов Иван Иванович	Электроснабжение	1909.68
7	Петрова Мария Сергеевна	Водоснабжение	364.00
8	Сидоров Алексей Петров...	Газоснабжение	481.34
9	ООО "СтройСервис"	Водоснабжение	1365.00
10	ООО "СтройСервис"	Электроснабжение	2452.80

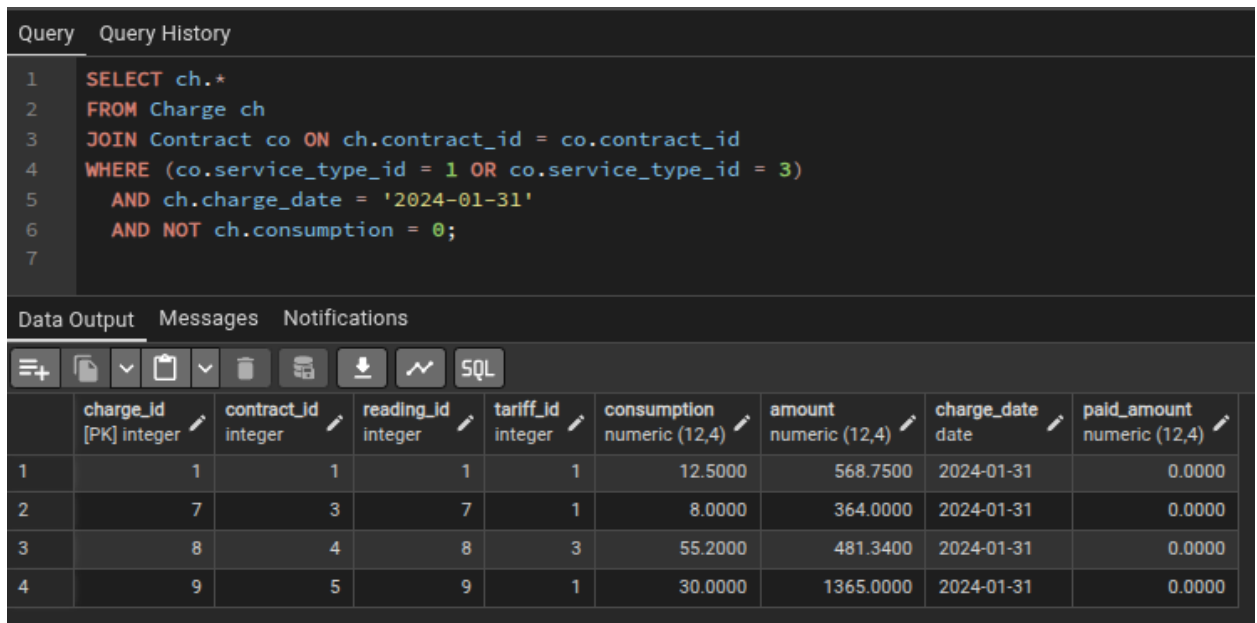
Рисунок 11. – Результат выполнения многотабличного запроса с вычисляемым полем и простым условием

6.1.4. Многотабличные запросы со сложными условиями

d1. Условия OR, AND, NOT:

Задача: вывести начисления по водоснабжению или газоснабжению за январь 2024 года, кроме начислений с нулевым потреблением.

Результат выполнения запроса представлен на рисунке 12.



The screenshot shows a database query interface. At the top, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying a SQL query. Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table of results. The table has 9 columns: charge_id, contract_id, reading_id, tariff_id, consumption, amount, charge_date, and paid_amount. The results are displayed in a table with 4 rows.

```
1 SELECT ch.*
2 FROM Charge ch
3 JOIN Contract co ON ch.contract_id = co.contract_id
4 WHERE (co.service_type_id = 1 OR co.service_type_id = 3)
5        AND ch.charge_date = '2024-01-31'
6        AND NOT ch.consumption = 0;
7
```

	charge_id [PK] integer	contract_id integer	reading_id integer	tariff_id integer	consumption numeric (12,4)	amount numeric (12,4)	charge_date date	paid_amount numeric (12,4)
1	1	1	1	1	12.5000	568.7500	2024-01-31	0.0000
2	7	3	7	1	8.0000	364.0000	2024-01-31	0.0000
3	8	4	8	3	55.2000	481.3400	2024-01-31	0.0000
4	9	5	9	1	30.0000	1365.0000	2024-01-31	0.0000

Рисунок 12. – Результат выполнения многотабличного запроса с условием OR, AND, NOT

d2. Условия AND, AND:

Задача: вывести начисления по электроэнергии с потреблением более 300 кВт·ч и суммой более 1500 руб.

Результат выполнения запроса представлен на рисунке 13.

Query

Query History

1

SELECT ch.*

2

FROM Charge ch

3

JOIN Contract co ON ch.contract_id = co.contract_id

4

WHERE co.service_type_id = 2

5

AND ch.consumption > 300

6

AND ch.amount > 1500;

Data Output

Messages

Notifications

≡

+

▼

▼

▼

▼

▼

▼

SQL

	charge_id [PK] integer	contract_id integer	reading_id integer	tariff_id integer	consumption numeric (12,4)	amount numeric (12,4)	charge_date date	paid_amount numeric (12,4)
1	4	2	4	2	310.0000	1810.4000	2024-01-31	0.0000
2	5	2	5	2	308.0000	1798.7200	2024-02-29	0.0000
3	6	2	6	2	327.0000	1909.6800	2024-03-31	0.0000
4	10	6	10	2	420.0000	2452.8000	2024-01-31	0.0000

Рисунок 13. – Результат выполнения многотабличного запроса с условием AND, AND

d3. Условие с оператором BETWEEN:

Задача: вывести показания счётчиков воды, снятые в период с января по март 2024 года.

Результат выполнения запроса представлен на рисунке 14.

Query

Query History

1

SELECT mr.*

2

FROM MeterReading mr

3

JOIN Meter m ON mr.meter_id = m.meter_id

4

JOIN Contract co ON m.contract_id = co.contract_id

5

WHERE co.service_type_id = 1

6

AND mr.reading_date BETWEEN '2024-01-01' AND '2024-03-31';

7

Data Output

Messages

Notifications

SQL

	reading_id [PK] integer	meter_id integer	reading_value numeric (12,4)	reading_date date
1	3	1	41.3000	2024-03-31
2	2	1	26.1000	2024-02-29
3	1	1	12.5000	2024-01-31
4	15	3	17.5000	2024-02-29
5	7	3	8.0000	2024-01-31
6	17	5	58.0000	2024-02-29
7	9	5	30.0000	2024-01-31

Рисунок 14. – Результат выполнения многотабличного запроса с оператором BETWEEN

d4. Условие с оператором NOT BETWEEN:

Задача: вывести платежи юридических лиц, совершённые вне периода с 01.01 по 28.02.2024.

Результат выполнения запроса представлен на рисунке 15.

Query

Query History

1

SELECT p.*

2

FROM Payment p

3

JOIN Client cl ON p.client_id = cl.client_id

4

WHERE cl.client_type = 'юридическое'

5

AND p.payment_date NOT BETWEEN '2024-01-01' AND '2024-02-28';

6

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑

🗄

⬇

📶

SQL

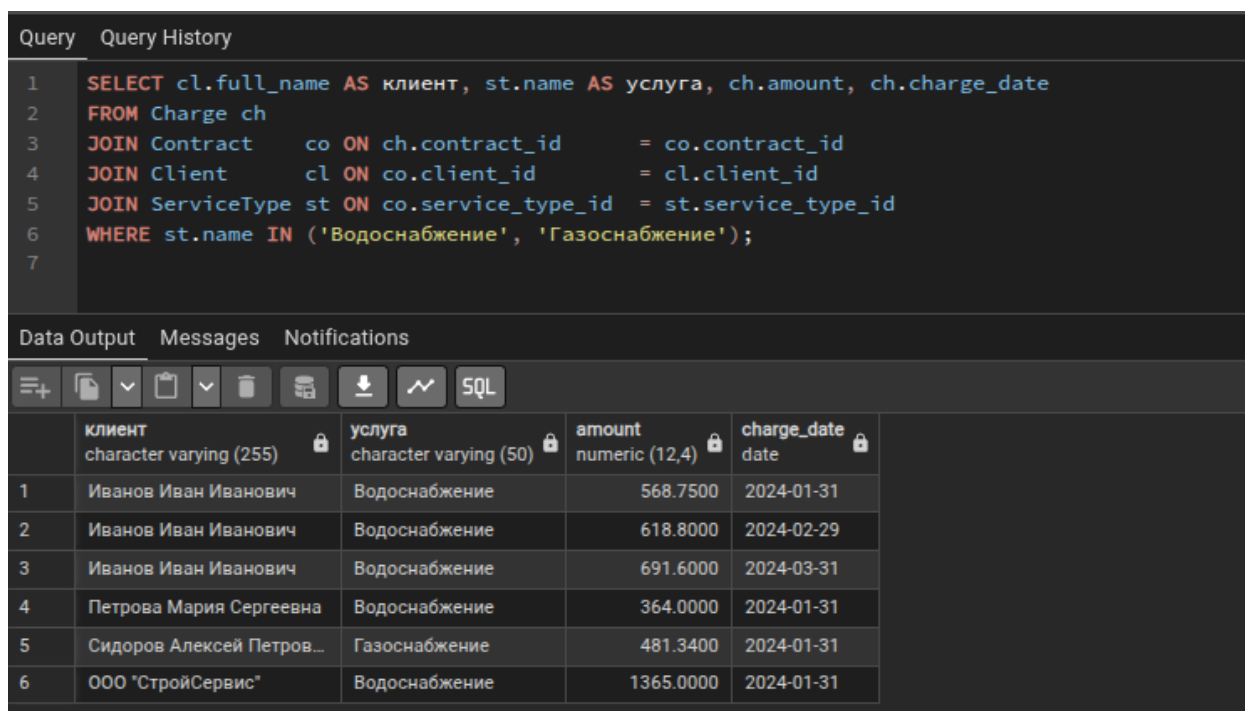
	payment_id [PK] integer	client_id integer	amount numeric (12,4)	payment_date date	note text
1	6	4	2500.0000	2024-04-05	Оплата за ма...

Рисунок 15. – Результат выполнения многотабличного запроса с оператором NOT BETWEEN

d5. Условие с оператором IN:

Задача: вывести ФИО клиентов и вид услуги по начислениям, относящимся к водоснабжению или газоснабжению.

Результат выполнения запроса представлен на рисунке 16.



The screenshot shows a database query interface. At the top, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying an SQL query. Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table of results. The table has four columns: 'клиент' (client), 'услуга' (service), 'amount', and 'charge_date'. The results are as follows:

	клиент character varying (255)	услуга character varying (50)	amount numeric (12,4)	charge_date date
1	Иванов Иван Иванович	Водоснабжение	568.7500	2024-01-31
2	Иванов Иван Иванович	Водоснабжение	618.8000	2024-02-29
3	Иванов Иван Иванович	Водоснабжение	691.6000	2024-03-31
4	Петрова Мария Сергеевна	Водоснабжение	364.0000	2024-01-31
5	Сидоров Алексей Петров...	Газоснабжение	481.3400	2024-01-31
6	ООО "СтройСервис"	Водоснабжение	1365.0000	2024-01-31

Рисунок 16. – Результат выполнения многотабличного запроса с оператором
IN

d6. Условие с оператором NOT IN:

Задача: вывести ФИО и тип клиентов, у которых нет ни одного платежа в базе данных.

Результат выполнения запроса представлен на рисунке 17.

Query	Query History
1	SELECT cl.full_name, cl.client_type
2	FROM Client cl
3	JOIN Contract co ON cl.client_id = co.client_id
4	WHERE cl.client_id NOT IN (
5	SELECT DISTINCT client_id
6	FROM Payment
7	);
8	

Data Output	Messages	Notifications
≡ + 📄 ▼ 📋 ▼ 🗑 📦 ⬇ ⤴ SQL		
	full_name character varying (255) 🔒	client_type character varying (20) 🔒
1	АО "ТехноПром"	юридическое
2	АО "ТехноПром"	юридическое

Рисунок 17. – Результат выполнения многотабличного запроса с оператором NOT IN

6.2. Запросы с использованием оконных функций

1. ROW_NUMBER

Задача: для каждого начисления указать его порядковый номер в истории начислений данного клиента (по дате).

Результат выполнения запроса представлен на рисунке 18.

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

12

SELECT

cl.full_name AS клиент,

ch.charge_date,

ch.amount,

ROW_NUMBER() OVER (

PARTITION BY co.client_id

ORDER BY ch.charge_date

) AS номер_начисления

FROM Charge ch

JOIN Contract co ON ch.contract_id = co.contract_id

JOIN Client cl ON co.client_id = cl.client_id;

Data Output

Messages

Notifications

≡

📄

▼

📋

🗑

🗑

📥

⬇

📈

SQL

	клиент character varying (255) 🔒	charge_date date 🔒	amount numeric (12,4) 🔒	номер_начисления bigint 🔒
1	Иванов Иван Иванович	2024-01-31	568.7500	1
2	Иванов Иван Иванович	2024-01-31	1810.4000	2
3	Иванов Иван Иванович	2024-02-29	1798.7200	3
4	Иванов Иван Иванович	2024-02-29	618.8000	4
5	Иванов Иван Иванович	2024-03-31	1909.6800	5
6	Иванов Иван Иванович	2024-03-31	691.6000	6
7	Петрова Мария Сергеевна	2024-01-31	364.0000	1
8	Сидоров Алексей Петров...	2024-01-31	481.3400	1
9	ООО "СтройСервис"	2024-01-31	1365.0000	1
10	ООО "СтройСервис"	2024-01-31	2452.8000	2

Рисунок 18. – Результат выполнения запроса с оконной функцией ROW_NUMBER

2. SUM OVER (накопительный итог)

Задача: Для каждого клиента вывести накопительную сумму начислений по дате.

Результат выполнения запроса представлен на рисунке 19.

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

12

13

14

SELECT

cl.full_name AS клиент,

ch.charge_date,

ch.amount,

SUM(ch.amount) OVER (

PARTITION BY co.client_id

ORDER BY ch.charge_date

ROWS UNBOUNDED PRECEDING

) AS накопительная_сумма

FROM Charge ch

JOIN Contract co ON ch.contract_id = co.contract_id

JOIN Client cl ON co.client_id = cl.client_id

ORDER BY cl.full_name, ch.charge_date;

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	клиент character varying (255) 🔒	charge_date date 🔒	amount numeric (12,4) 🔒	накопительная_сумма numeric 🔒
1	Иванов Иван Иванович	2024-01-31	1810.4000	2379.1500
2	Иванов Иван Иванович	2024-01-31	568.7500	568.7500
3	Иванов Иван Иванович	2024-02-29	1798.7200	4177.8700
4	Иванов Иван Иванович	2024-02-29	618.8000	4796.6700
5	Иванов Иван Иванович	2024-03-31	691.6000	7397.9500
6	Иванов Иван Иванович	2024-03-31	1909.6800	6706.3500
7	ООО "СтройСервис"	2024-01-31	1365.0000	1365.0000
8	ООО "СтройСервис"	2024-01-31	2452.8000	3817.8000
9	Петрова Мария Сергеевна	2024-01-31	364.0000	364.0000
10	Сидоров Алексей Петров...	2024-01-31	481.3400	481.3400

Рисунок 19. – Результат выполнения запроса с оконной функцией SUM()
OVER

3. RANK

Задача: Проранжировать клиентов по суммарному долгу (наибольший долг — ранг 1), включая клиентов без задолженности.

Результат выполнения запроса представлен на рисунке 20.

Query	Query History
1	SELECT
2	cl.full_name AS клиент,
3	COALESCE(SUM(ch.amount - ch.paid_amount), 0) AS общий_долг,
4	RANK() OVER (
5	ORDER BY COALESCE(SUM(ch.amount - ch.paid_amount), 0) DESC
6	) AS ранг
7	FROM Client cl
8	LEFT JOIN Contract co ON cl.client_id = co.client_id
9	LEFT JOIN Charge ch ON co.contract_id = ch.contract_id
10	GROUP BY cl.client_id, cl.full_name
11	ORDER BY ранг;
12	

Data Output	Messages	Notifications
≡ + 📄 ▼ 📋 ▼ 🗑️ 🗑️ 📦 ⬇️ 📈 SQL		
	клиент character varying (255)	общий_долг numeric
		ранг bigint
1	Иванов Иван Иванович	7397.9500
2	ООО "СтройСервис"	3817.8000
3	Сидоров Алексей Петров...	481.3400
4	Петрова Мария Сергеевна	364.0000
5	АО "ТехноПром"	0

Рисунок 20. – Результат выполнения запроса с оконной функцией RANK()

6.3. Запросы с вычисляемыми полями

1. Вычисляемое поле 1

Задача: вывести задолженность по каждому начислению (начислено минус оплачено).

Результат выполнения запроса представлен на рисунке 21.

Query

Query History

1

SELECT

2

mr.reading_id,

3

cl.full_name AS клиент,

4

st.name AS услуга,

5

mr.reading_value AS показание,

6

t.price_per_unit AS тариф,

7

ROUND(mr.reading_value * t.price_per_unit, 2) AS расчётная_сумма

8

FROM MeterReading mr

9

JOIN Meter m ON mr.meter_id = m.meter_id

10

JOIN Contract co ON m.contract_id = co.contract_id

11

JOIN Client cl ON co.client_id = cl.client_id

12

JOIN ServiceType st ON co.service_type_id = st.service_type_id

13

JOIN Tariff t ON co.service_type_id = t.service_type_id

14

WHERE t.valid_to IS NULL;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🔄

⬇️

📈

SQL

	reading_id integer	клиент character varying (255)	услуга character varying (50)	показание numeric (12,4)	тариф numeric (12,4)	расчётная_сумма numeric
1	1	Иванов Иван Иванович	Водоснабжение	12.5000	45.5000	568.75
2	2	Иванов Иван Иванович	Водоснабжение	26.1000	45.5000	1187.55
3	3	Иванов Иван Иванович	Водоснабжение	41.3000	45.5000	1879.15
4	4	Иванов Иван Иванович	Электроснабжение	310.0000	5.8400	1810.40
5	5	Иванов Иван Иванович	Электроснабжение	618.0000	5.8400	3609.12
6	6	Иванов Иван Иванович	Электроснабжение	945.0000	5.8400	5518.80
7	7	Петрова Мария Сергеевна	Водоснабжение	8.0000	45.5000	364.00
8	15	Петрова Мария Сергеевна	Водоснабжение	17.5000	45.5000	796.25
9	8	Сидоров Алексей Петров...	Газоснабжение	55.2000	8.7200	481.34
10	16	Сидоров Алексей Петров...	Газоснабжение	67.0000	8.7200	584.24
11	9	ООО "СтройСервис"	Водоснабжение	30.0000	45.5000	1365.00
12	17	ООО "СтройСервис"	Водоснабжение	58.0000	45.5000	2639.00
13	10	ООО "СтройСервис"	Электроснабжение	420.0000	5.8400	2452.80
14	18	ООО "СтройСервис"	Электроснабжение	830.0000	5.8400	4847.20
15	11	АО "ТехноПром"	Газоснабжение	72.4000	8.7200	631.33
16	12	АО "ТехноПром"	Электроснабжение	380.0000	5.8400	2219.20
17	13	Петрова Мария Сергеевна	Газоснабжение	18.6000	8.7200	162.19
18	14	Сидоров Алексей Петров...	Электроснабжение	500.0000	5.8400	2920.00

Рисунок 22. – Результат выполнения запроса с вычисляемым полем №2

6.4. Запросы с агрегатными функциями

1. Группировка данных и простая сортировка.

Задача: вывести общую сумму начислений, количество начислений и общую оплату по каждому виду услуги, отсортировав по убыванию суммы.

Результат выполнения запроса представлен на рисунке 23.

Query

Query History

1

SELECT

2

st.name AS услуга,

3

COUNT(ch.charge_id) AS кол_начислений,

4

SUM(ch.amount) AS итого_начислено,

5

SUM(ch.paid_amount) AS итого_оплачено

6

FROM Charge ch

7

JOIN Contract co ON ch.contract_id = co.contract_id

8

JOIN ServiceType st ON co.service_type_id = st.service_type_id

9

GROUP BY st.name

10

ORDER BY итого_начислено DESC;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑

📦

⬇

📈

SQL

	услуга character varying (50) 🔒	кол_начислений bigint 🔒	итого_начислено numeric 🔒	итого_оплачено numeric 🔒
1	Электроснабжение	4	7971.6000	0.0000
2	Водоснабжение	5	3608.1500	0.0000
3	Газоснабжение	1	481.3400	0.0000

Рисунок 23. – Результат выполнения запроса с агрегатными функциями и простой сортировкой

2. Группировка с условием и многоуровневая сортировка (3 поля).

Задача: вывести среднюю сумму начислений по типу клиента, клиенту и виду услуги, отобрав только тех, у кого есть задолженность.

Результат выполнения запроса представлен на рисунке 24.

Query

Query History

1

2

3

4

5

6

7

SELECT *

FROM Charge

WHERE amount > (

SELECT AVG(amount)

FROM Charge

);

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

📄

⬇️

📈

SQL

	charge_id [PK] integer	contract_id integer	reading_id integer	tariff_id integer	consumption numeric (12,4)	amount numeric (12,4)	charge_date date	paid_amount numeric (12,4)
1	4	2	4	2	310.0000	1810.4000	2024-01-31	0.0000
2	5	2	5	2	308.0000	1798.7200	2024-02-29	0.0000
3	6	2	6	2	327.0000	1909.6800	2024-03-31	0.0000
4	9	5	9	1	30.0000	1365.0000	2024-01-31	0.0000
5	10	6	10	2	420.0000	2452.8000	2024-01-31	0.0000

Рисунок 25. – Результат выполнения запроса с подзапросом, возвращающим единичное значение

2. Операции IN и NOT IN

Задача: вывести клиентов, у которых есть договоры, но нет ни одного платежа.

Результат выполнения запроса представлен на рисунке 26.

Query

Query History

1

SELECT *

2

FROM Client

3

WHERE client_id IN (

4

SELECT DISTINCT client_id

5

FROM Contract

6

)

7

AND client_id NOT IN (

8

SELECT DISTINCT client_id

9

FROM Payment

10

);

11

Data Output

Messages

Notifications

SQL

	client_id [PK] integer	client_type character varying (20)	full_name character varying (255)	address text	phone character varying (20)	email character varying (100)
1	5	юридическое	АО "ТехноПром"	ул. Заводская, ...	79134445566	techno@biz.ru

Рисунок 26. – Результат выполнения запроса с использованием операций IN и NOT IN

3. Ключевые слова ANY и ALL

Задача: вывести начисления, сумма которых больше любого платежа физических лиц, но меньше каждого платежа юридических лиц.

Результат выполнения запроса представлен на рисунке 27.

```
1 SELECT *
2 FROM Charge
3 WHERE amount > ANY (
4     SELECT p.amount
5     FROM Payment p
6     JOIN Client cl ON p.client_id = cl.client_id
7     WHERE cl.client_type = 'физическое'
8 )
9 AND amount < ALL (
10    SELECT p.amount
11    FROM Payment p
12    JOIN Client cl ON p.client_id = cl.client_id
13    WHERE cl.client_type = 'юридическое'
14 );
15
```

	charge_id [PK] integer	contract_id integer	reading_id integer	tariff_id integer	consumption numeric (12,4)	amount numeric (12,4)	charge_date date	paid_amount numeric (12,4)
1	1	1	1	1	12.5000	568.7500	2024-01-31	0.0000
2	2	1	2	1	13.6000	618.8000	2024-02-29	0.0000
3	3	1	3	1	15.2000	691.6000	2024-03-31	0.0000
4	4	2	4	2	310.0000	1810.4000	2024-01-31	0.0000
5	5	2	5	2	308.0000	1798.7200	2024-02-29	0.0000
6	6	2	6	2	327.0000	1909.6800	2024-03-31	0.0000
7	8	4	8	3	55.2000	481.3400	2024-01-31	0.0000
8	9	5	9	1	30.0000	1365.0000	2024-01-31	0.0000
9	10	6	10	2	420.0000	2452.8000	2024-01-31	0.0000

Рисунок 27. – Результат выполнения запроса с использованием ключевых слов ANY и ALL

4. Операции EXISTS и NOT EXISTS

Задача: вывести клиентов, у которых есть хотя бы одно начисление, но нет ни одного платежа.

Результат выполнения запроса представлен на рисунке 28.

The screenshot shows a database query editor with a dark theme. The top panel displays an SQL query using the `EXISTS` and `NOT EXISTS` operators. The query selects all columns from the `Client` table (`cl`) where there exists a contract (`co`) with a charge (`ch`) for the same client, and there does not exist a payment (`p`) for the same client.

```

1  SELECT *
2  FROM Client cl
3  WHERE EXISTS (
4      SELECT 1
5      FROM Contract co
6      JOIN Charge ch ON co.contract_id = ch.contract_id
7      WHERE co.client_id = cl.client_id
8  )
9  AND NOT EXISTS (
10     SELECT 1
11     FROM Payment p
12     WHERE p.client_id = cl.client_id
13 );
14

```

The bottom panel shows the 'Data Output' tab with a table of results. The table has 7 columns: `client_id` (integer), `client_type` (character varying (20)), `full_name` (character varying (255)), `address` (text), `phone` (character varying (20)), and `email` (character varying (100)). The first row of data shows a client with `client_id` 5, `client_type` 'юридическое', `full_name` 'АО "ТехноПром"', `address` 'ул. Заводская, ...', `phone` '79134445566', and `email` 'techno@biz.ru'.

	client_id [PK] integer	client_type character varying (20)	full_name character varying (255)	address text	phone character varying (20)	email character varying (100)
1	5	юридическое	АО "ТехноПром"	ул. Заводская, ...	79134445566	techno@biz.ru

Рисунок 28. – Результат выполнения запроса с использованием операций `EXISTS` и `NOT EXISTS`

6.6. Запрос на основе материализованного представления

Задача: создать материализованное представление, хранящее суммарную задолженность каждого клиента по каждой услуге, и выполнить к нему запрос.

Результат выполнения запроса представлен на рисунке 29.

QueryQuery History

```

1 DROP MATERIALIZED VIEW IF EXISTS mv_client_debt;
2
3 CREATE MATERIALIZED VIEW mv_client_debt AS
4 SELECT
5     cl.client_id,
6     cl.full_name,
7     cl.client_type,
8     st.name AS услуга,
9     SUM(ch.amount - ch.paid_amount) AS общий_долг,
10    COUNT(ch.charge_id) AS кол_начислений
11 FROM Client cl
12 JOIN Contract co ON cl.client_id = co.client_id
13 JOIN Charge ch ON co.contract_id = ch.contract_id
14 JOIN ServiceType st ON co.service_type_id = st.service_type_id
15 WHERE ch.paid_amount < ch.amount
16 GROUP BY cl.client_id, cl.full_name, cl.client_type, st.name;
17
18 SELECT
19     full_name AS клиент,
20     client_type AS тип,
21     услуга,
22     ROUND(общий_долг, 2) AS долг_руб
23 FROM mv_client_debt
24 ORDER BY общий_долг DESC;
25

```

Data OutputMessagesNotifications

+

📄

▼

📄

▼

🗑️

🔍

⬇️

📈

SQL

	клиент character varying (255)	тип character varying (20)	услуга character varying (50)	долг_руб numeric
1	Иванов Иван Иванович	физическое	Электроснабжение	5518.80
2	ООО "СтройСервис"	юридическое	Электроснабжение	2452.80
3	Иванов Иван Иванович	физическое	Водоснабжение	1879.15
4	ООО "СтройСервис"	юридическое	Водоснабжение	1365.00
5	АО "ТехноПром"	юридическое	Газоснабжение	630.93
6	Сидоров Алексей Петров...	физическое	Газоснабжение	481.34
7	Петрова Мария Сергеевна	физическое	Водоснабжение	364.00

Рисунок 29. – Результат выполнения запроса к материализованному представлению mv_client_debt

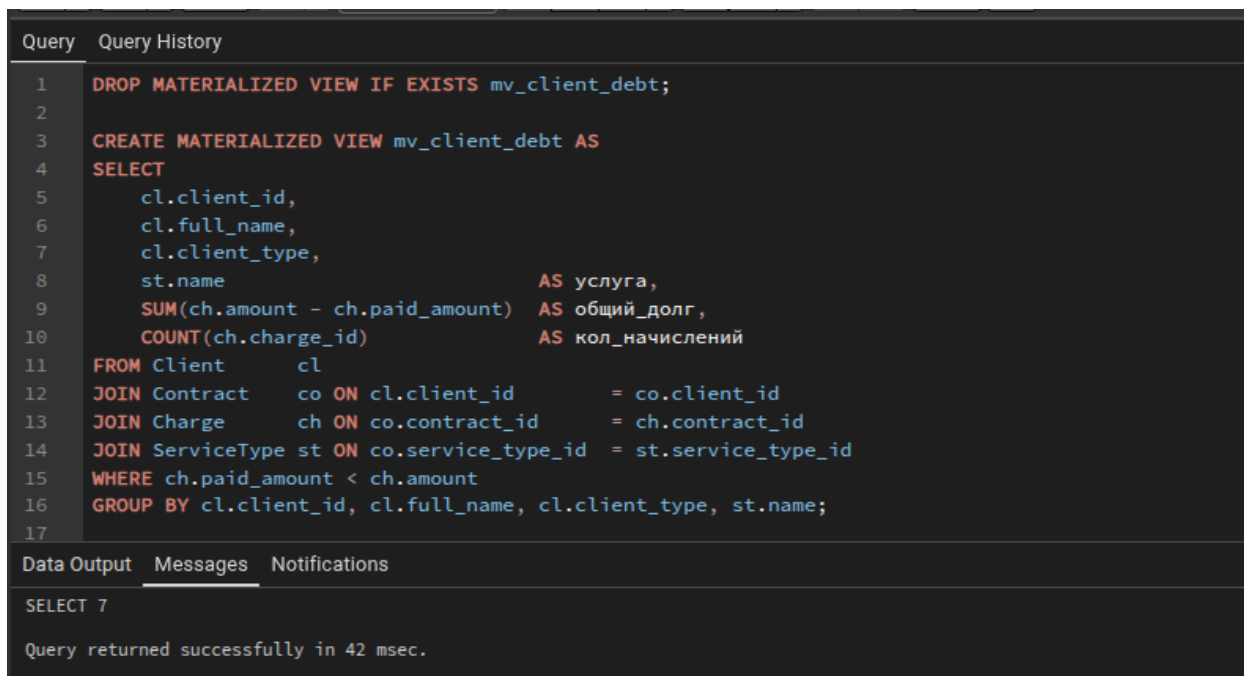
7. Разработка объектов БД

7.1. Разработка материализованных представлений

1. mv_client_debt

Назначение: хранит суммарную задолженность каждого клиента в разрезе видов услуг. Используется для быстрого формирования отчётов о задолженности без выполнения сложных JOIN-запросов.

Результат создания представления представлен на рисунке 30.



```
Query  Query History
1  DROP MATERIALIZED VIEW IF EXISTS mv_client_debt;
2
3  CREATE MATERIALIZED VIEW mv_client_debt AS
4  SELECT
5      cl.client_id,
6      cl.full_name,
7      cl.client_type,
8      st.name AS услуга,
9      SUM(ch.amount - ch.paid_amount) AS общий_долг,
10     COUNT(ch.charge_id) AS кол_начислений
11  FROM Client cl
12  JOIN Contract co ON cl.client_id = co.client_id
13  JOIN Charge ch ON co.contract_id = ch.contract_id
14  JOIN ServiceType st ON co.service_type_id = st.service_type_id
15  WHERE ch.paid_amount < ch.amount
16  GROUP BY cl.client_id, cl.full_name, cl.client_type, st.name;
17

Data Output  Messages  Notifications
SELECT 7
Query returned successfully in 42 msec.
```

Рисунок 30. – Создание материализованного представления mv_client_debt

Для проверки корректности работы представления выполнен следующий запрос.

Результат представлен на рисунке 31.

Query

Query History

1

2

3

4

5

6

7

8

SELECT

full_nameAS клиент,

client_typeAS тип,

услуга,

ROUND(общий_долг, 2) AS долг_руб

FROM mv_client_debt

ORDER BY общий_долг DESC;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🔍

⬇️

📈

SQL

	клиент character varying (255) 🔒	тип character varying (20) 🔒	услуга character varying (50) 🔒	долг_руб numeric 🔒
1	Иванов Иван Иванович	физическое	Электроснабжение	5518.80
2	ООО "СтройСервис"	юридическое	Электроснабжение	2452.80
3	Иванов Иван Иванович	физическое	Водоснабжение	1879.15
4	ООО "СтройСервис"	юридическое	Водоснабжение	1365.00
5	АО "ТехноПром"	юридическое	Газоснабжение	630.93
6	Сидоров Алексей Петров...	физическое	Газоснабжение	481.34
7	Петрова Мария Сергеевна	физическое	Водоснабжение	364.00

Рисунок 31. – Результат запроса к материализованному представлению mv_client_debt

2. mv_monthly_consumption

Назначение: хранит агрегированные данные о потреблении по каждому прибору учёта в разрезе месяцев. Применяется для анализа динамики потребления ресурсов.

Результат создания представления представлен на рисунке 32.

```
Query  Query History
1  CREATE MATERIALIZED VIEW mv_monthly_consumption AS
2  SELECT
3      m.meter_id,
4      st.name                                AS услуга,
5      DATE_TRUNC('month', mr.reading_date)   AS период,
6      COUNT(mr.reading_id)                  AS кол_показаний,
7      MAX(mr.reading_value) - MIN(mr.reading_value) AS потребление
8  FROM MeterReading mr
9  JOIN Meter      m ON mr.meter_id      = m.meter_id
10 JOIN Contract   co ON m.contract_id   = co.contract_id
11 JOIN ServiceType st ON co.service_type_id = st.service_type_id
12 GROUP BY m.meter_id, st.name, DATE_TRUNC('month', mr.reading_date);
13

Data Output  Messages  Notifications
SELECT 18
Query returned successfully in 57 msec.
```

Рисунок 32. – Создание материализованного представления
mv_monthly_consumption

Для проверки корректности работы представления выполнен следующий запрос.

Результат представлен на рисунке 33.

Query

Query History

1

SELECT *

2

FROM mv_monthly_consumption

3

ORDER BY период, meter_id;

4

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑️

🗑️

📦

⬇️

📈

SQL

	meter_id integer 🔒	услуга character varying (50) 🔒	период timestamp with time zone 🔒	кол_показаний bigint 🔒	потребление numeric 🔒
1	1	Водоснабжение	2024-01-01 00:00:00+06	1	0.0000
2	2	Электроснабжение	2024-01-01 00:00:00+06	1	0.0000
3	3	Водоснабжение	2024-01-01 00:00:00+06	1	0.0000
4	4	Газоснабжение	2024-01-01 00:00:00+06	1	0.0000
5	5	Водоснабжение	2024-01-01 00:00:00+06	1	0.0000
6	6	Электроснабжение	2024-01-01 00:00:00+06	1	0.0000
7	7	Газоснабжение	2024-01-01 00:00:00+06	1	0.0000
8	8	Электроснабжение	2024-01-01 00:00:00+06	1	0.0000
9	9	Газоснабжение	2024-01-01 00:00:00+06	1	0.0000
10	10	Электроснабжение	2024-01-01 00:00:00+06	1	0.0000
11	1	Водоснабжение	2024-02-01 00:00:00+06	1	0.0000
12	2	Электроснабжение	2024-02-01 00:00:00+06	1	0.0000
13	3	Водоснабжение	2024-02-01 00:00:00+06	1	0.0000
14	4	Газоснабжение	2024-02-01 00:00:00+06	1	0.0000
15	5	Водоснабжение	2024-02-01 00:00:00+06	1	0.0000
16	6	Электроснабжение	2024-02-01 00:00:00+06	1	0.0000
17	1	Водоснабжение	2024-03-01 00:00:00+06	1	0.0000
18	2	Электроснабжение	2024-03-01 00:00:00+06	1	0.0000

Рисунок 33. – Результат запроса к материализованному представлению
mv_monthly_consumption

7.2. Разработка курсоров

1. Курсор 1

Назначение: перебирает клиентов с ненулевым долгом и выводит уведомление с ФИО, типом клиента и суммой задолженности.

Результат выполнения курсора представлен на рисунке 34.

Query	Query History
1	DO \$\$
2	DECLARE
3	cur_debt CURSOR FOR
4	SELECT
5	cl.full_name,
6	cl.client_type,
7	SUM(ch.amount - ch.paid_amount) AS долг
8	FROM Client cl
9	JOIN Contract co ON cl.client_id = co.client_id
10	JOIN Charge ch ON co.contract_id = ch.contract_id
11	WHERE ch.paid_amount < ch.amount
12	GROUP BY cl.client_id, cl.full_name, cl.client_type;
13	rec RECORD;
14	BEGIN
15	OPEN cur_debt;
16	LOOP
17	FETCH cur_debt INTO rec;
18	EXIT WHEN NOT FOUND;
19	RAISE NOTICE 'Должник: % (%) – долг: % руб.',
20	rec.full_name, rec.client_type, ROUND(rec.долг, 2);
21	END LOOP;
22	CLOSE cur_debt;
23	END \$\$;
24	

Data Output	Messages	Notifications
NOTICE: Должник: Иванов Иван Иванович (физическое) – долг: 7397.95 руб.		
NOTICE: Должник: Петрова Мария Сергеевна (физическое) – долг: 364.00 руб.		
NOTICE: Должник: Сидоров Алексей Петрович (физическое) – долг: 481.34 руб.		
NOTICE: Должник: ООО "СтройСервис" (юридическое) – долг: 3817.80 руб.		
NOTICE: Должник: АО "ТехноПром" (юридическое) – долг: 630.93 руб.		
DO		
Query returned successfully in 49 msec.		

Рисунок 34. – Результат выполнения курсора №1 (перебор должников)

2. Курсор 2

Назначение: перебирает все показания, снятые в январе 2024 года, и выводит уведомление с ФИО клиента, видом услуги, значением и датой показания.

Результат выполнения курсора представлен на рисунке 35.

Query	Query History
1	DO \$\$
2	DECLARE
3	cur_jan CURSOR FOR
4	SELECT
5	cl.full_name AS клиент,
6	st.name AS услуга,
7	mr.reading_value,
8	mr.reading_date
9	FROM MeterReading mr
10	JOIN Meter m ON mr.meter_id = m.meter_id
11	JOIN Contract co ON m.contract_id = co.contract_id
12	JOIN Client cl ON co.client_id = cl.client_id
13	JOIN ServiceType st ON co.service_type_id = st.service_type_id
14	WHERE mr.reading_date BETWEEN '2024-01-01' AND '2024-01-31';
15	rec RECORD;
16	BEGIN
17	OPEN cur_jan;
18	LOOP
19	FETCH cur_jan INTO rec;
20	EXIT WHEN NOT FOUND;
21	RAISE NOTICE 'Клиент: %, услуга: %, показание: % от %',
22	rec.клиент, rec.услуга, rec.reading_value, rec.reading_date;
23	END LOOP;
24	CLOSE cur_jan;
25	END \$\$;
26	

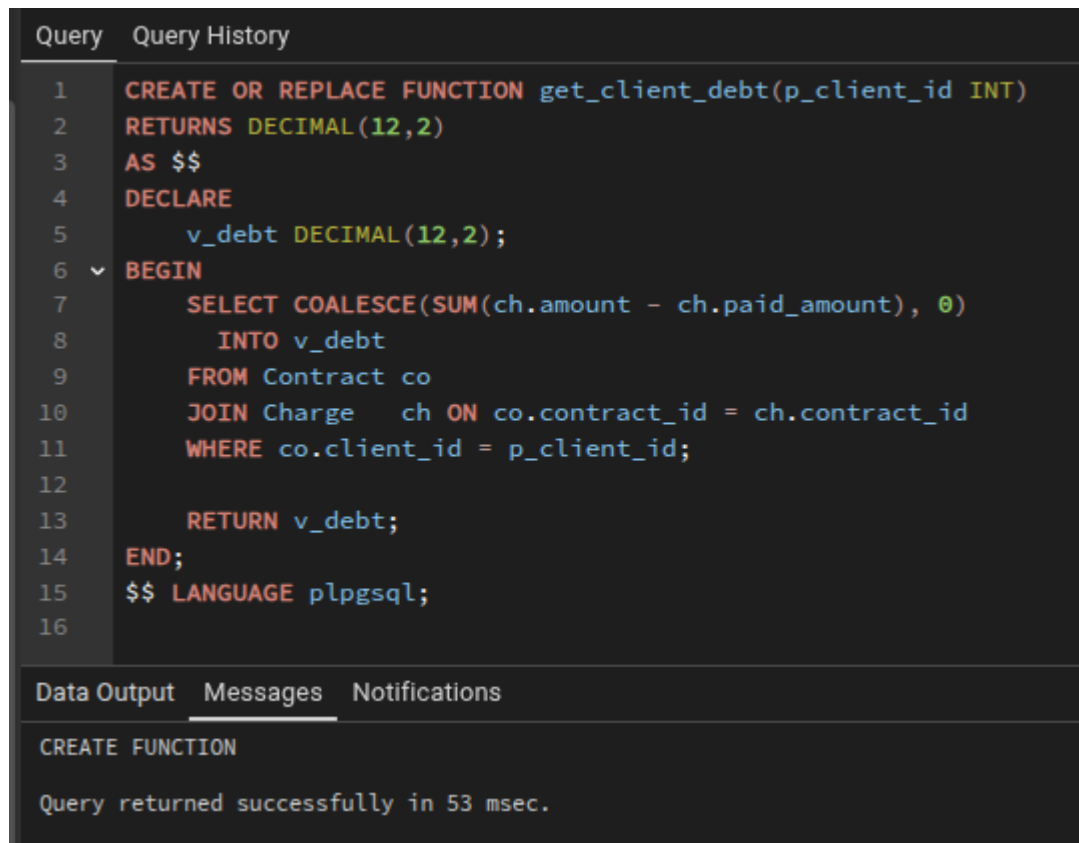
Data Output	Messages	Notifications
NOTICE:	Клиент:	Иванов Иван Иванович, услуга: Водоснабжение, показание: 12.5000 от 2024-01-31
NOTICE:	Клиент:	Иванов Иван Иванович, услуга: Электроснабжение, показание: 310.0000 от 2024-01-31
NOTICE:	Клиент:	Петрова Мария Сергеевна, услуга: Водоснабжение, показание: 8.0000 от 2024-01-31
NOTICE:	Клиент:	Сидоров Алексей Петрович, услуга: Газоснабжение, показание: 55.2000 от 2024-01-31
NOTICE:	Клиент:	000 "СтройСервис", услуга: Водоснабжение, показание: 30.0000 от 2024-01-31
NOTICE:	Клиент:	000 "СтройСервис", услуга: Электроснабжение, показание: 420.0000 от 2024-01-31
NOTICE:	Клиент:	АО "ТехноПром", услуга: Газоснабжение, показание: 72.4000 от 2024-01-31
NOTICE:	Клиент:	АО "ТехноПром", услуга: Электроснабжение, показание: 380.0000 от 2024-01-31
NOTICE:	Клиент:	Петрова Мария Сергеевна, услуга: Газоснабжение, показание: 18.6000 от 2024-01-31
NOTICE:	Клиент:	Сидоров Алексей Петрович, услуга: Электроснабжение, показание: 500.0000 от 2024-01-31
DO		
Query returned successfully in 48 msec.		

Рисунок 35. – Результат выполнения курсора №2 (показания за январь 2024)

7.3. Разработка функции

Назначение: принимает идентификатор клиента и возвращает суммарную сумму его задолженности по всем услугам с точностью до 0,01 руб.

Результат создания функции представлен на рисунке 36.



```
1 CREATE OR REPLACE FUNCTION get_client_debt(p_client_id INT)
2 RETURNS DECIMAL(12,2)
3 AS $$
4 DECLARE
5     v_debt DECIMAL(12,2);
6 BEGIN
7     SELECT COALESCE(SUM(ch.amount - ch.paid_amount), 0)
8     INTO v_debt
9     FROM Contract co
10    JOIN Charge ch ON co.contract_id = ch.contract_id
11    WHERE co.client_id = p_client_id;
12
13    RETURN v_debt;
14 END;
15 $$ LANGUAGE plpgsql;
16
```

Query returned successfully in 53 msec.

Рисунок 36. – Создание функции get_client_debt

Для проверки корректности работы функции выполнен следующий запрос.

Результат представлен на рисунке 37.

Query		Query History	
1	SELECT		
2	full_name	AS клиент,	
3	get_client_debt(client_id)	AS долг_руб	
4	FROM Client		
5	ORDER BY долг_руб	DESC;	
6			

Data Output		Messages	Notifications
≡+ 📄 ▼ 📋 ▼ 🗑 📦 ⬇ ⤴ SQL			
	клиент character varying (255) 🔒		долг_руб numeric 🔒
1	Иванов Иван Иванович		7397.95
2	ООО "СтройСервис"		3817.80
3	АО "ТехноПром"		630.93
4	Сидоров Алексей Петров...		481.34
5	Петрова Мария Сергеевна		364.00

Рисунок 37. – Результат вызова функции get_client_debt

7.4. Разработка процедуры

Назначение: принимает идентификатор клиента, сумму и дату платежа; регистрирует платёж в таблице Payment и автоматически распределяет его по неоплаченным начислениям в хронологическом порядке (принцип FIFO).

Результат создания процедуры представлен на рисунке 38.


```
Query  Query History
1  CREATE OR REPLACE PROCEDURE distribute_payment(
2      p_client_id  INT,
3      p_amount     DECIMAL(12,2),
4      p_date       DATE DEFAULT CURRENT_DATE
5  )
6  LANGUAGE plpgsql AS $$
7  DECLARE
8      v_remaining DECIMAL(12,2) := p_amount;
9      rec         RECORD;
10 BEGIN
11     INSERT INTO Payment (client_id, amount, payment_date)
12     VALUES (p_client_id, p_amount, p_date);
13
14     FOR rec IN
15         SELECT ch.charge_id,
16                ch.amount - ch.paid_amount AS долг
17         FROM Charge  ch
18         JOIN Contract co ON ch.contract_id = co.contract_id
19         WHERE co.client_id = p_client_id
20               AND ch.paid_amount < ch.amount
21         ORDER BY ch.charge_date
22     LOOP
23         EXIT WHEN v_remaining <= 0;
24
25         IF v_remaining >= rec.долг THEN
26             UPDATE Charge
27             SET paid_amount = amount
28             WHERE charge_id = rec.charge_id;
29             v_remaining := v_remaining - rec.долг;
30         ELSE
31             UPDATE Charge
32             SET paid_amount = paid_amount + v_remaining
33             WHERE charge_id = rec.charge_id;
34             v_remaining := 0;
35         END IF;
36     END LOOP;
37
38     RAISE NOTICE 'Платёж % руб. распределён. Остаток: % руб.',
39     p_amount, v_remaining;
40 END;
41 $$;
42
```

Data Output Messages Notifications

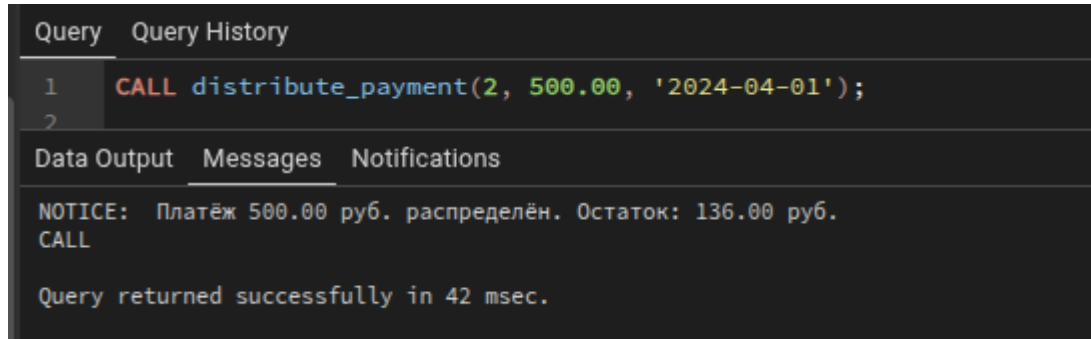
CREATE PROCEDURE

Query returned successfully in 42 msec.

Рисунок 38. – Создание процедуры distribute_payment

Для проверки корректности работы процедуры выполнен следующий вызов.

Результат представлен на рисунке 39.



```
Query  Query History
1  CALL distribute_payment(2, 500.00, '2024-04-01');
2
Data Output  Messages  Notifications
NOTICE: Платёж 500.00 руб. распределён. Остаток: 136.00 руб.
CALL
Query returned successfully in 42 msec.
```

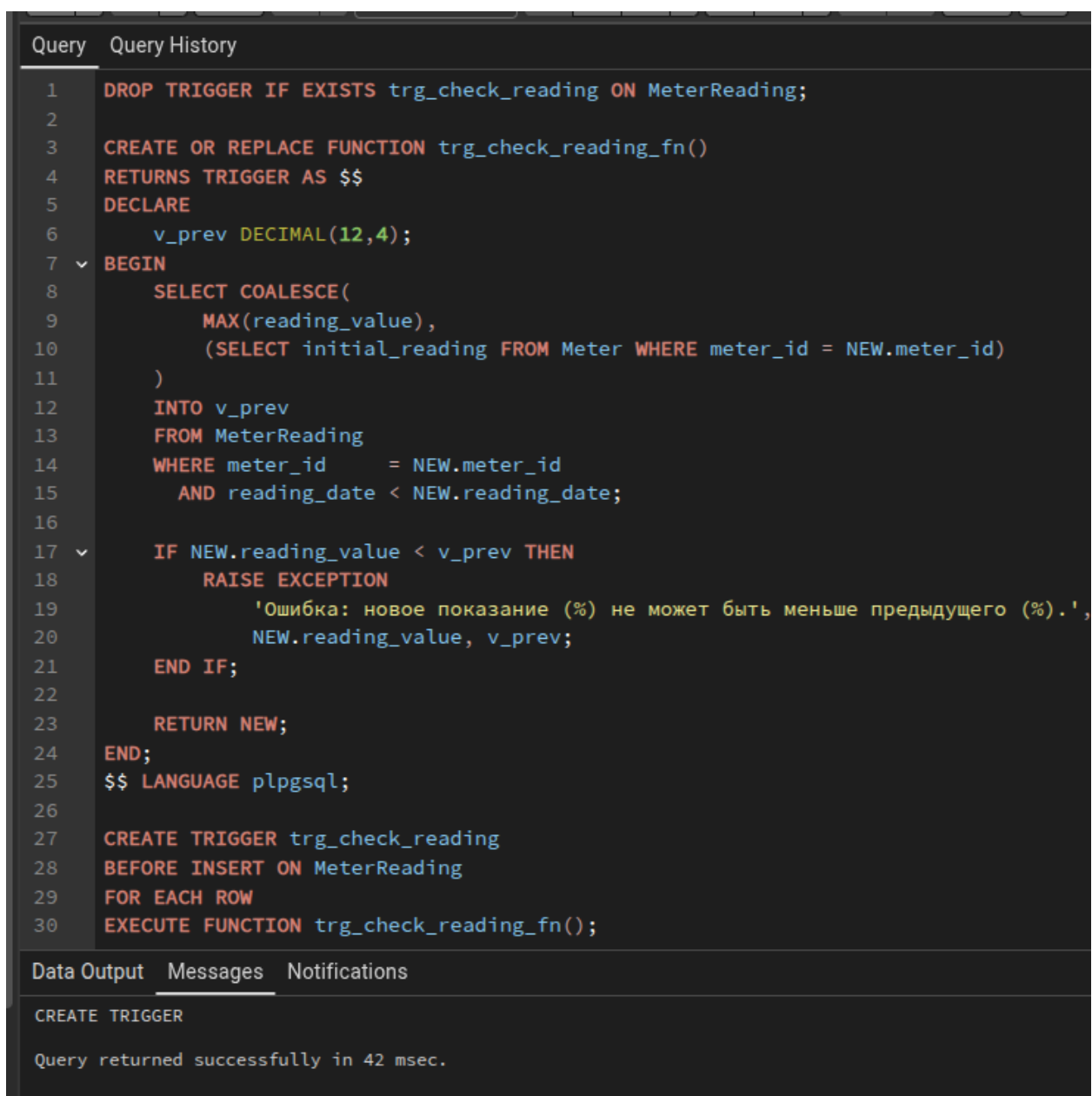
Рисунок 39. – Результат вызова процедуры distribute_payment

7.5. Разработка триггеров

1. проверка показания (trg_check_reading)

Назначение: срабатывает перед вставкой нового показания и проверяет, что его значение не меньше предыдущего зафиксированного показания данного прибора. При нарушении условия генерируется исключение и операция отменяется.

Результат создания триггера представлен на рисунке 40.



```
1 DROP TRIGGER IF EXISTS trg_check_reading ON MeterReading;
2
3 CREATE OR REPLACE FUNCTION trg_check_reading_fn()
4 RETURNS TRIGGER AS $$
5 DECLARE
6     v_prev DECIMAL(12,4);
7 BEGIN
8     SELECT COALESCE(
9         MAX(reading_value),
10        (SELECT initial_reading FROM Meter WHERE meter_id = NEW.meter_id)
11    )
12    INTO v_prev
13    FROM MeterReading
14    WHERE meter_id = NEW.meter_id
15    AND reading_date < NEW.reading_date;
16
17    IF NEW.reading_value < v_prev THEN
18        RAISE EXCEPTION
19            'Ошибка: новое показание (%) не может быть меньше предыдущего (%)',
20            NEW.reading_value, v_prev;
21    END IF;
22
23    RETURN NEW;
24 END;
25 $$ LANGUAGE plpgsql;
26
27 CREATE TRIGGER trg_check_reading
28 BEFORE INSERT ON MeterReading
29 FOR EACH ROW
30 EXECUTE FUNCTION trg_check_reading_fn();
```

CREATE TRIGGER

Query returned successfully in 42 msec.

Рисунок 40. – Создание триггера trg_check_reading

2. автоматическое начисление (trg_create_charge)

Назначение: срабатывает после успешной вставки показания и автоматически формирует запись начисления: вычисляет потребление как разность текущего и предыдущего показания, определяет действующий тариф и рассчитывает сумму к оплате.

Результат создания триггера представлен на рисунке 41.

```
1 DROP TRIGGER IF EXISTS trg_create_charge ON MeterReading;
2
3 CREATE OR REPLACE FUNCTION trg_create_charge_fn()
4 RETURNS TRIGGER AS $$
5 DECLARE
6     v_prev_val    DECIMAL(12,4);
7     v_contract_id INT;
8     v_service_id  INT;
9     v_tariff_id   INT;
10    v_price        DECIMAL(12,4);
11    v_consumption  DECIMAL(12,4);
12 BEGIN
13     SELECT COALESCE(
14         MAX(reading_value),
15         (SELECT initial_reading FROM Meter WHERE meter_id = NEW.meter_id)
16     )
17     INTO v_prev_val
18     FROM MeterReading
19     WHERE meter_id = NEW.meter_id
20     AND reading_date < NEW.reading_date;
21
22     SELECT m.contract_id, co.service_type_id
23     INTO v_contract_id, v_service_id
24     FROM Meter m
25     JOIN Contract co ON m.contract_id = co.contract_id
26     WHERE m.meter_id = NEW.meter_id;
27
28     SELECT tariff_id, price_per_unit
29     INTO v_tariff_id, v_price
30     FROM Tariff
31     WHERE service_type_id = v_service_id
32     AND valid_from <= NEW.reading_date
33     AND (valid_to IS NULL OR valid_to >= NEW.reading_date)
34     ORDER BY valid_from DESC
35     LIMIT 1;
36
37     v_consumption := NEW.reading_value - v_prev_val;
38
39     INSERT INTO Charge (contract_id, reading_id, tariff_id,
40         consumption, amount, charge_date)
41     VALUES (v_contract_id, NEW.reading_id, v_tariff_id,
42         v_consumption,
43         ROUND(v_consumption * v_price, 2),
44         NEW.reading_date);
45
46     RETURN NEW;
47 END;
48 $$ LANGUAGE plpgsql;
49
50 CREATE TRIGGER trg_create_charge
51 AFTER INSERT ON MeterReading
52 FOR EACH ROW
53 EXECUTE FUNCTION trg_create_charge_fn();
54
```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 40 msec.

Рисунок 41. – Создание триггера trg_create_charge

Для проверки корректности работы триггеров были выполнены тестовые запросы.

Тест 1: корректная вставка показания (ожидается успешное добавление и автоматическое создание начисления).

Результат представлен на рисунке 42.



The screenshot shows a database query window with a dark theme. The 'Query' tab is active, displaying a SQL script with line numbers 1 through 6. The script consists of a BEGIN statement, an INSERT INTO statement for the MeterReading table, a SELECT statement, and a ROLLBACK statement. Below the query, the 'Data Output' tab is active, showing the word 'ROLLBACK' and a message: 'Query returned successfully in 41 msec.' The 'Messages' and 'Notifications' tabs are also visible but empty.

```
1 BEGIN;  
2 INSERT INTO MeterReading (meter_id, reading_value, reading_date)  
3 VALUES (1, 55.0000, '2024-04-30');  
4 SELECT * FROM Charge ORDER BY charge_id DESC LIMIT 1;  
5 ROLLBACK;  
6
```

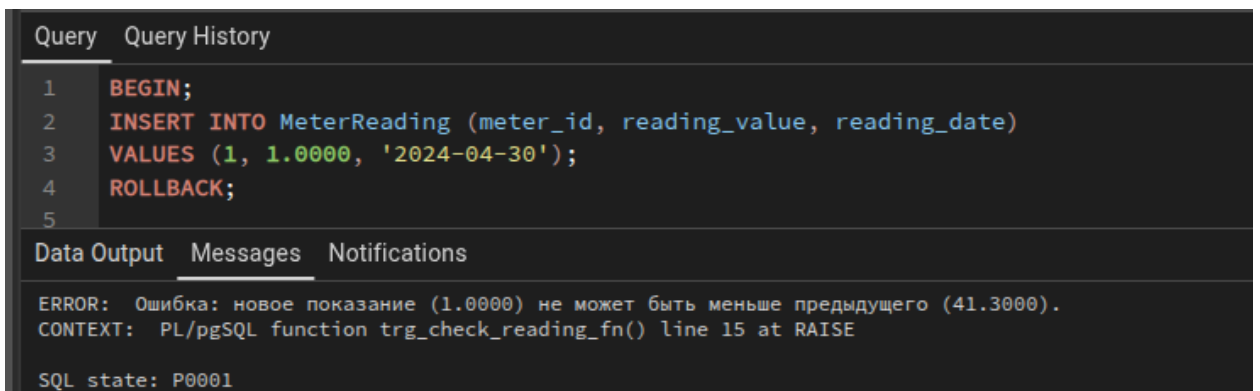
ROLLBACK

Query returned successfully in 41 msec.

Рисунок 42. – Успешная вставка показания и автоматическое начисление

Тест 2: вставка показания, значение которого меньше предыдущего (ожидается ошибка).

Результат представлен на рисунке 43.



The screenshot shows a database query window with a dark theme. The 'Query' tab is active, displaying a SQL script with line numbers 1 through 5. The script consists of a BEGIN statement, an INSERT INTO statement for the MeterReading table, and a ROLLBACK statement. Below the query, the 'Data Output' tab is active, showing an error message: 'ERROR: Ошибка: новое показание (1.0000) не может быть меньше предыдущего (41.3000). CONTEXT: PL/pgSQL function trg_check_reading_fn() line 15 at RAISE'. The 'Messages' and 'Notifications' tabs are also visible but empty.

```
1 BEGIN;  
2 INSERT INTO MeterReading (meter_id, reading_value, reading_date)  
3 VALUES (1, 1.0000, '2024-04-30');  
4 ROLLBACK;  
5
```

ERROR: Ошибка: новое показание (1.0000) не может быть меньше предыдущего (41.3000).
CONTEXT: PL/pgSQL function trg_check_reading_fn() line 15 at RAISE

SQL state: P0001

Рисунок 43. – Ошибка при вставке некорректного показания

8. Ограничения целостности БД

Для подтверждения корректности работы механизмов целостности были выполнены тестовые транзакции.

Тест 1: нарушение ссылочной целостности — попытка добавить договор с несуществующим клиентом.

Результат представлен на рисунке 44.



The screenshot shows a SQL query execution window with a dark background. The top section, titled 'Query' and 'Query History', contains a SQL script with five lines: 1. BEGIN; 2. INSERT INTO Contract (client_id, service_type_id, start_date) 3. VALUES (9999, 1, '2024-01-01'); 4. ROLLBACK; 5. The bottom section, titled 'Data Output', 'Messages', and 'Notifications', displays an error message: 'ERROR: insert or update on table "contract" violates foreign key constraint "contract_client_id_fkey" Key (client_id)=(9999) is not present in table "client".' Below the error message, it shows 'SQL state: 23503' and 'Detail: Key (client_id)=(9999) is not present in table "client".'

```
1 BEGIN;
2 INSERT INTO Contract (client_id, service_type_id, start_date)
3 VALUES (9999, 1, '2024-01-01');
4 ROLLBACK;
5
```

ERROR: insert or update on table "contract" violates foreign key constraint "contract_client_id_fkey"
Key (client_id)=(9999) is not present in table "client".

SQL state: 23503
Detail: Key (client_id)=(9999) is not present in table "client".

Рисунок 44. – Ошибка нарушения ссылочной целостности

Тест 2: нарушение ограничения CHECK — попытка добавить платёж с отрицательной суммой.

Результат представлен на рисунке 45.

```
Query  Query History
1  BEGIN;
2  INSERT INTO Payment (client_id, amount, payment_date)
3  VALUES (1, -500.00, '2024-01-15');
4  ROLLBACK;
5

Data Output  Messages  Notifications
ERROR: new row for relation "payment" violates check constraint "payment_amount_check"
Failing row contains (8, 1, -500.0000, 2024-01-15, null).

SQL state: 23514
Detail: Failing row contains (8, 1, -500.0000, 2024-01-15, null).
```

Рисунок 45. – Ошибка нарушения ограничения CHECK

Тест 3: нарушение семантической целостности через триггер — попытка добавить показание меньше предыдущего.

Результат представлен на рисунке 46.

```
Query  Query History
1  BEGIN;
2  INSERT INTO MeterReading (meter_id, reading_value, reading_date)
3  VALUES (1, 0.0001, '2024-04-30');
4  ROLLBACK;
5

Data Output  Messages  Notifications
ERROR: Ошибка: новое показание (0.0001) не может быть меньше предыдущего (41.3000).
CONTEXT: PL/pgSQL function trg_check_reading_fn() line 15 at RAISE
SQL state: P0001
```

Рисунок 46. – Ошибка нарушения семантической целостности (триггер)

ЗАКЛЮЧЕНИЕ

В ходе выполнения расчётно-графической работы по дисциплине «Системы управления базами данных» была успешно спроектирована и реализована база данных для автоматизации учёта оплаты коммунальных услуг (водоснабжение, электроснабжение, газоснабжение), предоставляемых физическим и юридическим лицам.

В процессе работы были последовательно решены все поставленные задачи:

- Анализ предметной области позволил выделить ключевые сущности, атрибуты и бизнес-правила, определяющие логику функционирования системы учёта.
- Концептуальное и логическое проектирование обеспечило создание непротиворечивой модели данных, отражающей взаимосвязи между клиентами, договорами, услугами, приборами учёта, показаниями, тарифами, начислениями и платежами.
- Физическая реализация в СУБД PostgreSQL включала создание девяти взаимосвязанных таблиц, определение типов данных, установку ограничений целостности (PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE) и создание индексов для оптимизации производительности.
- Разработка разнообразных SQL-запросов продемонстрировала возможности извлечения и анализа данных: от простой выборки до сложных запросов с оконными функциями, подзапросами, агрегацией и использованием материализованных представлений.
- Создание программных объектов БД (курсоры, хранимые функции, процедуры, триггеры) позволило реализовать бизнес-логику непосредственно на уровне базы данных, обеспечив автоматизацию расчётов, распределения платежей, контроля хронологии показаний и обновления статусов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Малков, О. Б. Работа с СУБД PostgreSQL : учеб. пособие / О. Б. Малков, М. П. Маркова, М. В. Девятерикова ; Минобрнауки России, Ом. гос. техн. ун-т. – Омск : Изд-во ОмГТУ, 2023.
2. Моргунов, Е. П. PostgreSQL. Основы языка SQL: учеб. пособие / Е. П. Моргунов; под ред. Е. В. Рогова, П. В. Лузанова. — СПб.: БХВ-Петербург, 2018. — 336 с.: ил.